

The logo consists of a white rounded square with the word "Kesh" in a bold, dark blue sans-serif font centered within it.

Kesh

Documentation technique

Manuel administrateur

Installation, configuration et exploitation de Kesh dans un
environnement de production

Version **0.12.1** | 2026-10-07

Cible release : **v0.12**

<https://github.com/guycorbaz/kesh>

Table des matières

1	À propos de ce manuel	5
1.1	Public cible	5
1.2	Pré-requis de lecture	5
1.3	Conventions typographiques	5
2	Présentation de Kesh	6
2.1	Objectif du logiciel	6
2.2	Public cible du produit	6
2.3	Architecture technique haute	7
2.4	Licence	7
3	Prérequis système	7
3.1	Système d'exploitation	7
3.2	Logiciels requis	8
3.3	Ressources matérielles minimales	8
3.4	Ports réseau	8
3.4.1	Changer le port d'écoute (conflit port 80, ex. Synology DSM Web Station)	9
4	Installation	10
4.1	Méthode recommandée : Docker Compose	10
4.1.1	Étape 1 — Préparer l'environnement	10
4.1.2	Étape 2 — Récupérer le <code>docker-compose.yml</code>	10
4.1.3	Étape 3 — Créer le fichier <code>.env</code>	10
4.1.4	Étape 4 — Démarrer les services	12
4.1.5	Étape 5 — Vérifier l'installation	12
4.2	Configuration du reverse proxy HTTPS	12
4.3	Alternative : déploiement via Caddy	13
4.4	Alternative : déploiement via Traefik (avec firewall applicatif)	14
4.4.1	Compose Traefik en sidecar	14
4.4.2	Middlewares firewall applicatif	15
4.4.3	Plugins WAF avancés (optionnel, recommandé Internet-exposed)	16
4.4.4	Considérations sécurité spécifiques	17
4.5	Alternative : déploiement sur Synology DSM	17
4.5.1	Prérequis Synology DSM	17
4.5.2	Étape 1 — Préparer la structure de répertoires	18
4.5.3	Étape 2 — Récupérer <code>docker-compose.yml</code> et <code>.env</code>	18
4.5.4	Étape 3 — Importer le projet dans Container Manager	19
4.5.5	Étape 4 — Configurer le reverse proxy DSM (Application Portal)	20

4.5.6	Étape 5 — Vérification post-installation	21
4.5.7	Limitations Container Manager DSM	21
5	Configuration	21
5.1	Variables d'environnement complètes	21
5.1.1	Import de factures depuis un dossier	27
5.1.2	Monitoring	27
5.2	Configuration des logs fichier	27
5.3	Configuration MariaDB	28
5.3.1	Paramètres recommandés	28
5.3.2	Initialisation manuelle de la base	29
5.4	Configuration multi-tenant	30
5.4.1	Concept de <i>company</i>	30
5.4.2	Isolation des données	30
5.4.3	Cas d'usage fiduciaire	30
6	Premier démarrage	31
6.1	Création du compte administrateur initial (onboarding self-service)	31
6.1.1	Procédure (nouveau flow recommandé v0.1.2+)	31
6.1.2	Procédure alternative — bootstrap déclaratif via <code>.env</code>	31
6.2	Récupération de mot de passe par email (SMTP)	32
6.2.1	Variables de configuration	33
6.2.2	Exemples de providers	33
6.2.3	Fonctionnement et garanties	34
6.2.4	Test après configuration (recommandé)	34
6.2.5	Limitations connues	35
6.3	Envoi de factures par e-mail	35
6.3.1	Activation	35
6.3.2	Identité d'expédition	36
6.3.3	Garanties et limites	36
6.4	Modèles d'e-mail	36
6.5	Rappels débiteurs (réglages)	37
6.5.1	Niveaux de rappel	37
6.5.2	Période de grâce	37
6.6	J'ai oublié mon mot de passe administrateur (recovery break-glass)	38
6.6.1	Procédure step-by-step	38
6.6.2	Matrice de comportement au boot — vue d'ensemble	39
6.7	Déploiement LAN HTTP-only (sans HTTPS) — v0.1.3+	39
6.7.1	Décision : HTTPS ou HTTP-only ?	40
6.7.2	Procédure HTTP-only (<code>KESH_COOKIE_SECURE=false</code>)	40
6.7.3	Valeurs acceptées et fail-fast	41
6.8	Configuration de la company via l'onboarding	41
6.9	Choix du plan comptable	42

6.10	Configuration des exercices comptables	42
6.11	Création des utilisateurs et rôles	43
7	Sauvegarde et restauration	44
7.1	Importance de la sauvegarde	44
7.2	Stratégie recommandée	44
7.3	Les PDF des factures émises	44
7.4	Script de sauvegarde mariadb-dump	45
7.5	Stockage offsite	46
7.6	Restauration depuis sauvegarde	46
7.7	Test de restauration périodique (OLICo Art. 10 al. 1)	47
7.8	Backup natif sur Synology DSM (Hyper Backup + Snapshot Replication)	48
7.8.1	Hyper Backup — backup incrémental + offsite intégré	48
7.8.2	Snapshot Replication — recovery instantané point-in-time	52
7.8.3	Stratégie 3-2-1 sur DSM	54
7.9	Export/import d'installation via l'interface Kesh	54
7.9.1	Exporter (sauvegarde complète)	55
7.9.2	Importer (restaurer la même installation)	55
7.9.3	Quelle méthode de sauvegarde choisir ?	58
8	Mise à jour de Kesh	58
8.1	Process de release Kesh	59
8.2	Suivre les notifications de release	59
8.3	Procédure de mise à jour standard	59
8.3.1	Passer à la 0.13.0 : le compose et le fichier <code>.env</code>	60
8.3.2	Changer un mot de passe MariaDB	65
8.4	Mise à jour majeure (changement de version X)	66
8.5	Rollback en cas d'échec	66
9	Sécurité	67
9.1	HTTPS obligatoire en production	67
9.2	Gestion des secrets	67
9.3	Authentification JWT + refresh tokens	67
9.4	Clés API (PAT) — accès programmatique	68
9.5	RBAC et permissions	70
9.6	Journal d'audit (audit_log)	70
9.7	Conformité OLICo Art. 9 — intégrité des supports modifiables	71
9.8	Sécurité réseau (firewall)	73
9.9	Mises à jour de sécurité du système hôte	73
10	Monitoring et logs	73
10.1	Logs applicatifs	73

10.2 Centralisation des logs	74
10.3 Supervision (état de santé)	74
10.4 Alerting recommandé	74
11 Conformité légale suisse	75
11.1 Code des obligations Art. 957a — Tenue de la comptabilité	75
11.2 Code des obligations Art. 958f — Conservation 10 ans	75
11.3 Rétention des rappels débiteurs (dossier de recouvrement)	76
11.4 Ordonnance OLICo (RS 221.431)	76
11.5 LSCSE (signature électronique qualifiée)	77
11.6 Loi sur la TVA (LTVA)	77
11.7 Documentation à conserver	79
12 Dépannage	80
12.1 Kesh ne démarre pas	80
12.2 Erreurs 401 en cascade côté frontend	81
12.3 Performance lente sur de gros datasets	81
12.4 Reset du mot de passe administrateur (break-glass)	81
12.5 Données corrompues détectées	82
13 Maintenance	82
13.1 Rotation des logs	82
13.2 Maintenance MariaDB	83
13.3 Vérification périodique de l'intégrité (OLICo Art. 10)	83
14 Annexes	84
14.1 Liste complète des variables d'environnement	84
14.2 Opérations en ligne de commande	84
14.3 Référence des ports	84
14.4 Glossaire	84
14.5 Liens externes	85
14.6 Support et communauté	85

1 À propos de ce manuel

Ce manuel s'adresse aux administrateurs système, responsables DevOps et fiduciaires souhaitant déployer et exploiter Kesh dans un environnement de production. Il couvre l'ensemble du cycle de vie opérationnel d'une instance Kesh : installation, configuration, sauvegarde, mise à jour, sécurité, conformité légale suisse et dépannage.

1.1 Public cible

Ce document est rédigé pour :

- Les administrateurs système (Linux ou macOS) responsables de l'hébergement d'une instance Kesh pour une ou plusieurs PME.
- Les fiduciaires souhaitant proposer Kesh comme solution comptable à leurs clients en mode self-hosting.
- Les développeurs internes d'une PME prenant en charge l'auto-hébergement de leur instance.
- Les responsables sécurité ou conformité devant valider la posture Kesh face aux exigences du Code des obligations suisse (CO Art. 957a, 958f) et de l'Ordonnance OLICo (RS 221.431).

1.2 Pré-requis de lecture

Le lecteur est supposé familier avec :

- Les concepts de base de Docker et Docker Compose (images, volumes, networks).
- L'administration d'un système Linux (CLI, systemd, gestion de paquets, permissions de fichiers).
- Les principes d'une base de données relationnelle (MariaDB/MySQL).
- Les concepts de reverse proxy HTTPS (Nginx, Traefik ou équivalent).
- Les notions de base de cryptographie appliquée (TLS, certificats, secrets).

1.3 Conventions typographiques

- `commande shell` : commande à exécuter dans un terminal.
- `/chemin/fichier` : chemin vers un fichier ou répertoire sur le système.
- `Ctrl+S` : combinaison de touches clavier.
- Les boîtes colorées : *Note* (information), *Astuce*, *Attention* (avertissement), *Danger* (risque critique).

i Note

Ce manuel décrit Kesh version 0.12.1. Pour la dernière version, consultez <https://github.com/guycorbaz/kesh>. Voir la section finale « À propos de cette version » pour les modalités de mise à jour.

2 Présentation de Kesh

2.1 Objectif du logiciel

Kesh est un **logiciel de comptabilité et de gestion** destiné aux indépendants, petites et moyennes entreprises (PME) et associations en Suisse. Il fournit un système complet conforme aux exigences du Code des obligations suisse, incluant :

- **Comptabilité en partie double** avec les plans comptables suisses PME, indépendant et association.
- **Facturation QR Bill 2.2** conforme au standard SIX Interbank Clearing (norme suisse 2020+).
- **Import bancaire** CAMT.053 (ISO 20022) et CSV multi-encodage avec profils banque réutilisables.
- **Réconciliation automatique et manuelle** des transactions bancaires avec règles d'affectation.
- **Rapports comptables** : bilan, compte de résultat, balance, journal.
- **Exports** légaux : PDF par rapport, ZIP global de souveraineté avec SHA-256 d'intégrité.
- **Multi-utilisateurs** avec contrôle d'accès basé sur les rôles (RBAC).
- **Multi-tenant** : isolation stricte des données par société (*company_id*).
- **Multilingue** : interface utilisateur en français, allemand, italien et anglais.

2.2 Public cible du produit

Kesh est conçu pour :

- Les **entreprises individuelles** et les **sociétés de personnes** avec un chiffre d'affaires supérieur à CHF 500'000 (régime « comptabilité complète » selon CO Art. 957 al. 1).
- Les **entreprises individuelles** et **sociétés de personnes** avec un chiffre d'affaires inférieur à CHF 500'000, qui peuvent légalement se limiter au régime « recettes-dépenses + patrimoine » (CO Art. 957 al. 2). *Kesh ne propose pas ce régime simplifié : il tient une comptabilité en partie double, ce qui est admis mais plus exigeant que ce que la loi leur impose.*
- Les **personnes morales** (SA, Sàrl, coopératives, associations, fondations).

- Les **fiduciaires** gérant plusieurs dossiers clients. *Un dossier = une instance : l'interface ne crée pas de seconde société et ne permet pas d'en changer.*

Kesh n'est pas conçu pour les grands groupes soumis à l'audit ordinaire (Art. 727 CO, seuils 20/40/250 selon Art. 727 al. 1 ch. 2 CO) ni aux entités cotées en bourse soumises à IFRS / Swiss GAAP RPC obligatoires.

2.3 Architecture technique haute

- **Backend** : Rust 1.96+ (édition 2024) avec le framework HTTP Axum et le client SQL SQLx.
- **Frontend** : SvelteKit 2 avec Svelte 5 et TypeScript, bundler Vite, CSS via Tailwind 4.
- **Base de données** : MariaDB 10.11+ (compatibilité MySQL 8 maintenue, mais MariaDB recommandée pour les optimisations spécifiques utilisées).
- **Déploiement** : Docker Compose, image officielle [gcorbaz/kesh:latest](#) sur Docker Hub.
- **Authentification** : JWT (HS256) avec refresh tokens et rotation, durée d'expiration courte (15 min) compensée par rafraîchissement automatique côté frontend.
- **Multi-tenant** : claim `company_id` dans le JWT et scoping strict sur toutes les requêtes API et toutes les requêtes SQL (cf. Story Epic 7-1 « audit multi-tenant scoping refactor »).

2.4 Licence

Kesh est distribué sous [licence EUPL 1.2](#) (European Union Public Licence). Cette licence est compatible GPL et permet l'usage commercial, la modification et la redistribution sous réserve de conserver la licence et les mentions d'attribution.

3 Prérequis système

3.1 Système d'exploitation

Kesh est officiellement supporté sur :

- **Linux** : Debian 12+, Ubuntu 22.04 LTS+, Fedora 38+, RHEL 9+, Arch Linux (rolling).
- **macOS** : 13 Ventura+ (architecture Intel ou Apple Silicon).

Le déploiement est officiellement supporté sur Linux uniquement. macOS est utilisable pour le développement et l'évaluation, mais non recommandé en production.

Le déploiement sur Windows (avec WSL2) est possible mais non officiellement testé.

3.2 Logiciels requis

- **Docker** ≥ 24.0 (Docker Engine ou Docker Desktop).
- **Docker Compose** ≥ 2.20 (plugin officiel [docker compose](#)).
- Un **reverse proxy HTTPS** en production (Nginx, Caddy, Traefik, HAProxy, ou équivalent).
- Un système de **sauvegarde MariaDB** (mariadb-dump cron, mariabackup, ou solution tierce).
- Optionnel : **Git** pour cloner le dépôt et suivre les mises à jour.

3.3 Ressources matérielles minimales

Ressource	Minimum	Recommandé
CPU	1 vCPU	2–4 vCPU
RAM	1 Go	4 Go
Stockage	5 Go (DB + logs)	50 Go (conservation 10 ans incluse)
Réseau	1 Mbps stable	10 Mbps + IPv4 publique
Disque	SSD recommandé pour DB	SSD obligatoire pour DB

i Note

Le stockage est dominé par l'*audit_log* et les pièces justificatives. Comptez environ **50–200 Mo par exercice** pour une PME standard. Pour une conservation OLICo 10 ans, prévoyez **1–3 Go** en moyenne.

3.4 Ports réseau

Port	Protocole	Usage	Exposition
443	HTTPS	Accès utilisateur via reverse proxy	Publique (Internet)
80	HTTP	API Axum backend (interne)	Localhost uniquement
5173	HTTP	SvelteKit dev server (dev seul)	Localhost uniquement
3306	MySQL/MariaDB	Connexion base de données	Voir ci-dessous

! Attention

Ne jamais exposer le port 80 (API Axum) ni le port 3306 (MariaDB) directement sur Internet. Toujours passer par un reverse proxy HTTPS qui termine TLS et fait suivre vers `localhost`.

Le port 3306 n'est pas publié par `docker-compose.yml` (depuis la 0.13.0) : Kesh atteint MariaDB par le réseau interne de Docker (hôte `mariadb`), et l'accès à la base passe par `docker compose exec mariadb`. Ne le publiez pas : **un port publié par Docker contourne le pare-feu de l'hôte** — Docker insère ses propres règles `iptables` avant celles d'UFW, si bien que `ufw default deny incoming` (§9.8) ne le ferme pas. Pour un dépannage ponctuel, le compose porte en commentaire une forme liée au seul loopback ("`127.0.0.1:3306:3306`"), à recommander ensuite. Avec `docker-compose.prod.yml` (Synology), la base n'est pas dans le compose : c'est la vôtre, et son port 3306 reste limité à `localhost` ou au réseau privé.

3.4.1 Changer le port d'écoute (conflit port 80, ex. Synology DSM Web Station)

Le port `80` par défaut peut entrer en conflit avec un service existant sur l'hôte — typiquement **Synology DSM Web Station** qui occupe également ce port, ou un `nginx` local, un `IIS`, etc. Quatre options d'override existent, selon votre contexte :

1. **Garder le port applicatif 80 mais remapper côté host** (recommandé) — dans `docker-compose.prod, dev.` changer le mapping "`127.0.0.1:80:80`" vers "`127.0.0.1:8080:80`" (ou tout autre `HOST_PORT` libre). L'application reste sur 80 côté container ; URL d'accès : `http://localhost:8080`.
2. **Changer le port applicatif lui-même** — dans `.env`, décommenter et adapter :

```
KESH_PORT=3000
```

puis aligner le mapping `docker-compose.*.yml` sur "`127.0.0.1:3000:3000`". URL d'accès : `http://localhost:3000`.

3. **Container sur une IP dédiée (Docker macvlan/ipvlan)** — configurer le container avec sa propre IP via réseau `macvlan` ; le mapping host devient inutile (le container répond directement sur son `IP:80`). Procédure spécifique à votre infrastructure réseau.
4. **Mode `cargo run` natif (hors Docker) sur Linux non-root** — Linux n'autorise pas les utilisateurs non-root à bind un port `<1024` (*security feature*). Dans ce contexte, set **obligatoirement** `KESH_PORT=3000` (ou un autre port `≥ 1024`) dans votre `.env` local. Alternatives possibles mais déconseillées : lancer avec `sudo`, ou ajouter la *capability* via `sudo setcap 'cap_net_bind_serv`. Hors Docker, posez aussi `KESH_ADMIN_BACKUP_DIR` sur un dossier inscriptible par l'utilisateur qui lance Kesh (son défaut `/data/backup` ne l'est pas) : sans quoi tout import d'installation est refusé (§7.9).

! Astuce

Sur Synology DSM, le port 80 est par défaut redirigé vers DSM Web Station. Sans override, le container Kesh ne pourra pas démarrer («port already in use»). La méthode (1) «remap côté host» est la plus simple pour cohabiter — par exemple `"127.0.0.1:8080:80"` + reverse proxy DSM vers `localhost:8080`.

4 Installation

4.1 Méthode recommandée : Docker Compose

Kesh est livré sous forme d'image Docker. Le déploiement standard utilise Docker Compose.

4.1.1 Étape 1 — Préparer l'environnement

Créer un répertoire dédié pour l'instance Kesh :

```
sudo mkdir -p /opt/kesh
sudo chown $USER:$USER /opt/kesh
cd /opt/kesh
```

4.1.2 Étape 2 — Récupérer le `docker-compose.yml`

Télécharger le fichier de référence depuis le dépôt officiel :

```
curl -fsSL
https://raw.githubusercontent.com/guycorbaz/kesh/main/docker-compose.yml \
-o docker-compose.yml
```

Ce fichier désigne l'image publiée `gcorbaz/kesh:latest` : `docker compose up -d` la télécharge, aucune construction depuis les sources n'est nécessaire.

4.1.3 Étape 3 — Créer le fichier `.env`

Copier le fichier d'exemple et l'adapter :

```
curl -fsSL
https://raw.githubusercontent.com/guycorbaz/kesh/main/.env.example \
-o .env
```

Éditer `.env` et renseigner au minimum les variables suivantes (cf. section 5.1 pour la liste complète).

KESH_JWT_SECRET est obligatoire et se génère : générez le secret (`openssl rand -hex 32`) et collez-le à la place du placeholder ; vérifiez : `grep KESH_JWT_SECRET .env` ne doit montrer ni `GENERATE_ME`, ni `change-me`, ni valeur entre chevrons `<...>`. Sans secret, Kesh refuse de démarrer, y compris avec `docker-compose.yml`, dont le défaut `change-me...` est refusé à dessein ; le placeholder `<GENERATE_ME: ...>` laissé tel quel est refusé depuis la 0.13.0. Les variables `KESH_ADMIN_*` restent optionnelles (onboarding `/setup`). Avec `docker-compose.yml`, `DATABASE_URL` est **composée** des `MARIADB_*` par le compose : la ligne `DATABASE_URL` de `.env` y est ignorée (elle sert `docker-compose.prod.yml`).

Remplacez chaque valeur entre chevrons `<...>` : recopiée telle quelle, elle fait refuser le démarrage (depuis la 0.13.0) pour `KESH_JWT_SECRET` et `KESH_ADMIN_PASSWORD`.

Les deux mots de passe MariaDB sont obligatoires avec `docker-compose.yml` (depuis la 0.13.0) : sur une **installation neuve**, générez chacun par `openssl rand -hex 32` — en hexadécimal, car une sortie base64 peut contenir `/`, qui casse une `DATABASE_URL`. Sans eux, **Compose refuse de démarrer**, au terminal, en nommant la variable manquante — une seule à la fois, pas toujours la même : posez les deux avant de relancer. Une valeur entre chevrons recopiée, en revanche, est acceptée par Compose et par MariaDB : Kesh **avertit au démarrage** quand c'est le mot de passe applicatif (`MARIADB_PASSWORD`, qu'il lit dans `DATABASE_URL`) ; **rien ne signale un mot de passe root resté au gabarit** — Kesh ne le voit pas. Relisez ces deux lignes.

MariaDB ne lit ces deux valeurs qu'à la création de la base (premier démarrage, volume vide) : les changer ensuite dans `.env` ne change pas le mot de passe enregistré, et Kesh échoue alors à se connecter (erreur 1045). Une base **déjà créée** avec la chaîne gabarit littérale — `<mot de passe fort>` **OU** `<mot de passe utilisateur applicatif>`, recopiées d'une version antérieure de ce manuel — garde ce mot de passe : ne le remplacez pas dans `.env`. De même, une base créée avant la 0.13.0 sans mot de passe posé garde les anciens défauts `kesh_dev_root` et `kesh_dev` : écrivez-les dans `.env`, jamais une valeur neuve (§8.3.1). Dans les deux cas, changez-le par la procédure du §8.3.2 (`ALTER USER` d'abord, `.env` ensuite).

```
# Secret JWT - générer avec : openssl rand -hex 32 (minimum 32 octets)
KESH_JWT_SECRET=<secret aleatoire, min. 32 octets, sans change-me ni
GENERATE_ME>

# Optionnel - sans elles, l'administrateur se crée à l'écran /setup
# KESH_ADMIN_USERNAME=admin
# KESH_ADMIN_PASSWORD=<mot de passe fort, min. 12 caracteres>

# Mots de passe MariaDB : OBLIGATOIRES avec docker-compose.yml, lus à
# la création de la base seulement. Chacun : openssl rand -hex 32
MARIADB_ROOT_PASSWORD=<sortie de openssl rand -hex 32>
MARIADB_PASSWORD=<autre sortie de openssl rand -hex 32>
MARIADB_DATABASE=kesh
MARIADB_USER=kesh
```

```
# URL de connexion DB : pour docker-compose.prod.yml seulement
# (docker-compose.yml la compose lui-même). Mot de passe hexadécimal.
DATABASE_URL=mysql://kesh:<MARIADB_PASSWORD>@mariadb:3306/kesh
```

i Note

KESH_PUBLIC_BASE_URL (base des liens de réinitialisation par email) n'est requise que si vous activez le recovery par email (KESH_FEATURE_FORGOT_PASSWORD=true, cf. §5.1).

X Danger

Ne jamais committer le fichier `.env` dans un dépôt Git. Il contient les secrets de production. Ajoutez-le à `.gitignore`.

4.1.4 Étape 4 — Démarrer les services

```
docker compose up -d
```

Vérifier que les services sont démarrés :

```
docker compose ps
docker compose logs -f kesh-api
```

Au premier démarrage, Kesh exécute automatiquement les migrations de schéma DB. Cela peut prendre 10–30 secondes selon la machine.

4.1.5 Étape 5 — Vérifier l'installation

Tester l'accès à l'API :

```
curl http://localhost/health
# Attendu :
{"status":"ok","db":true,"version":"<version>","forgotPasswordEnabled":false}
```

4.2 Configuration du reverse proxy HTTPS

En production, Kesh **doit** être accessible via HTTPS uniquement. Voici un exemple minimal avec Nginx :

```
# /etc/nginx/sites-available/kesh
server {
    listen 443 ssl http2;
    server_name compta.exemple.ch;
```

```
ssl_certificate
/etc/letsencrypt/live/compta.exemple.ch/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/compta.exemple.ch/privkey.pem;

# Strong cipher suite
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers on;

# Headers de securite
add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

location / {
    proxy_pass http://localhost;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;

    # Pour les uploads de pieces comptables et imports CAMT.053
    client_max_body_size 50M;

    # Timeouts adaptes pour les operations longues (exports ZIP)
    proxy_read_timeout 300s;
}

server {
    listen 80;
    server_name compta.exemple.ch;
    return 301 https://$host$request_uri;
}
```

Activer la configuration :

```
sudo ln -s /etc/nginx/sites-available/kesh /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

4.3 Alternative : déploiement via Caddy

Caddy gère automatiquement les certificats Let's Encrypt et représente une alternative plus simple :

```
# /etc/caddy/Caddyfile
compta.exemple.ch {
    reverse_proxy localhost {
        header_up X-Forwarded-Proto https
    }
    encode gzip
    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains"
        X-Content-Type-Options "nosniff"
        X-Frame-Options "DENY"
    }
}
```

4.4 Alternative : déploiement via Traefik (avec firewall applicatif)

Traefik est particulièrement adapté quand Kesh est exposé sur Internet : il intègre nativement des middlewares de *firewall applicatif* (rate limiting, IP whitelist, headers de sécurité) et supporte des plugins communautaires plus avancés (CrowdSec pour la détection d'intrusion, ModSecurity-CRS pour des règles WAF type OWASP). La configuration ci-dessous suppose Traefik 3.x déployé en sidecar dans le même Docker Compose que Kesh.

4.4.1 Compose Traefik en sidecar

Ajouter le service Traefik à votre `docker-compose.yml` (ou utiliser un `docker-compose.override.yml` dédié) :

```
services:
  traefik:
    image: traefik:v3.2
    container_name: kesh-traefik
    restart: unless-stopped
    command:
      # Entrypoints HTTP/HTTPS (HTTP redirige vers HTTPS).
      - --entrypoints.web.address=:80
      - --entrypoints.web.http.redirections.entryPoint.to=websecure
      - --entrypoints.web.http.redirections.entryPoint.scheme=https
      - --entrypoints.websecure.address=:443
      # Certificats Let's Encrypt automatiques (challenge HTTP-01).
      - --certificatesresolvers.le.acme.email=admin@exemple.ch
      - --certificatesresolvers.le.acme.storage=/letsencrypt/acme.json
      - --certificatesresolvers.le.acme.httpchallenge.entrypoint=web
      # Provider Docker - lit les labels sur les autres services.
      - --providers.docker=true
      - --providers.docker.exposedbydefault=false
```

```

    # Logs format JSON pour ingestion ultérieure (Loki, ELK, etc.).
    - --accesslog=true
    - --accesslog.format=json
    - --log.level=INFO
ports:
  - "80:80"
  - "443:443"
volumes:
  - /var/run/docker.sock:/var/run/docker.sock:ro
  - traefik-letsencrypt:/letsencrypt
networks:
  - kesh-internal

kesh-api:
  # ... votre config kesh-api existante ...
  # NE PAS publier le port 80 sur l'hôte (Traefik y accède via réseau Docker).
  expose:
    - "80"
  networks:
    - kesh-internal
  labels:
    - traefik.enable=true
    # Router HTTPS + cert Let's Encrypt.
    - traefik.http.routers.kesh.rule=Host(`compta.exemple.ch`)
    - traefik.http.routers.kesh.entrypoints=websecure
    - traefik.http.routers.kesh.tls.certresolver=le
    - traefik.http.services.kesh.loadbalancer.server.port=80
    # Chaîne de middlewares firewall applicatif (cf. ci-dessous).
    -
    traefik.http.routers.kesh.middlewares=kesh-ratelimit@docker,kesh-headers@docker

volumes:
  traefik-letsencrypt:

networks:
  kesh-internal:
    driver: bridge

```

4.4.2 Middlewares firewall applicatif

Définir les middlewares via labels sur le service `kesh-api` (ou dans un `dynamic.yml` dédié) :

```

labels:
  # ... labels routeur ci-dessus ...

  # 1. Rate limiting global - défense contre brute-force et scraping.

```

```

# Limite moyenne 50 req/s par IP, burst 100. Défense en
# profondeur du rate-limit applicatif Kesh (KESH_RATE_LIMIT_*).
- traefik.http.middlewares.kesh-ratelimit.ratelimit.average=50
- traefik.http.middlewares.kesh-ratelimit.ratelimit.burst=100
-
traefik.http.middlewares.kesh-ratelimit.ratelimit.sourcecriterion.ipstrategy.depth=1

# 2. Headers de sécurité (durcissement OWASP).
- traefik.http.middlewares.kesh-headers.headers.stsSeconds=31536000
-
traefik.http.middlewares.kesh-headers.headers.stsIncludeSubdomains=true
- traefik.http.middlewares.kesh-headers.headers.stsPreload=true
- traefik.http.middlewares.kesh-headers.headers.contentTypeNosniff=true
- traefik.http.middlewares.kesh-headers.headers.framedeny=true
-
traefik.http.middlewares.kesh-headers.headers.referrerPolicy=strict-origin-when-cross-origin
-
traefik.http.middlewares.kesh-headers.headers.permissionsPolicy=geolocation=(),microphone=()

# 3. (OPTIONNEL) IP whitelist pour endpoints sensibles - exemple
# pour bloquer /api/v1/_test/* (qui n'existe qu'en KESH_TEST_MODE
# mais on défend en profondeur). À adapter à vos IPs admin.
# -
traefik.http.middlewares.kesh-admin-only.ipallowlist.sourcerange=192.168.1.0/24,2001:db8::0/24

```

4.4.3 Plugins WAF avancés (optionnel, recommandé Internet-exposed)

Pour un firewall applicatif complet face à Internet, deux plugins Traefik communautaires sont éprouvés :

CrowdSec — détection d'intrusion collaborative CrowdSec analyse les logs Traefik en temps réel, détecte les scans/scrapes/brute-force, et partage les IPs malveillantes avec une CTI communautaire (équivalent crowd-sourced de *Fail2Ban* + base de réputation IP globale).

```

# Activer le plugin dans la commande Traefik (ajouter dans le `command:`
# ci-dessus) :
-
--experimental.plugins.crowdsec-bouncer.modulename=github.com/maxlerebourg/crowdsec-bouncer
- --experimental.plugins.crowdsec-bouncer.version=v1.3.5

# Puis sur kesh-api, ajouter le middleware :
labels:
-
traefik.http.middlewares.kesh-crowdsec.plugin.crowdsec-bouncer.enabled=true
-
traefik.http.middlewares.kesh-crowdsec.plugin.crowdsec-bouncer.crowdsecApiKey=<API_KEY>
# Puis chaîner dans la liste des middlewares du routeur :

```

```
traefik.http.routers.kesh.middlewares=kesh-crowdsec@docker,kesh-ratelimit@docker,kesh-hea
```

Installation locale CrowdSec : voir crowdsec.net. Inscription gratuite à la CTI communautaire pour partager les détections.

ModSecurity / OWASP Core Rule Set (CRS) — règles WAF traditionnelles Pour les organisations soumises à audit conformité (RGPD, LPD Suisse, exigences clients), un WAF basé sur OWASP CRS détecte les patterns d'attaque classiques (SQL injection, XSS, path traversal, etc.). Plusieurs intégrations sont disponibles :

- **Coraza** (plugin Traefik officiel WASM) — github.com/jcchavez/coraza-http-wasm. Engine WAF Go natif, performance élevée, règles CRS supportées.
- **ModSecurity v3 + Nginx** en frontal de Traefik — option plus complexe mais plus complète si déjà utilisé en interne.

4.4.4 Considérations sécurité spécifiques

- **Sock Docker exposé** (`/var/run/docker.sock`) : Traefik y accède en lecture seule (`:ro`). En haute exigence sécurité, préférer le `docker-proxy` (image `tecnativa/docker-socket-proxy`) qui filtre les endpoints Docker API accessibles à Traefik.
- **Renouvellement Let's Encrypt** : Traefik gère automatiquement le renouvellement 30 jours avant expiration. Surveiller via `docker logs kesh-traefik | grep acme`.
- **Backup de `acme.json`** : ce fichier contient la clé privée TLS — l'inclure dans votre stratégie de backup (volume `traefik-letsencrypt` ci-dessus).
- **Dashboard Traefik désactivé par défaut** dans cette config (pas de label `api`, pas d'option `--api`). Si vous l'activez en debug, protégez-le impérativement avec une auth + IP whitelist.

4.5 Alternative : déploiement sur Synology DSM

Synology DSM (DiskStation Manager) 7.2+ inclut le paquet *Container Manager* (anciennement *Docker*) qui permet de déployer Kesh sans utiliser le CLI Docker. Cette section couvre le scénario standard d'un déploiement Kesh sur un NAS Synology personnel ou de PME (DS220+, DS920+, DS1522+, DS1821+, ou modèles supérieurs avec ≥ 2 GiB RAM).

4.5.1 Prérequis Synology DSM

- **DSM** ≥ 7.2 (Container Manager n'est pas disponible sur DSM 6.x).
- **Modèle NAS** compatible Docker — vérifier sur la liste officielle Synology : modèles avec processeur Intel/AMD x86_64 (DS220+, DS720+, DS920+, DS1522+, DS1821+, etc.). Les modèles ARM (DS120j, DS220j, DS418, etc.) ne supportent pas Container Manager.
- **RAM** : minimum 2 GiB (4 GiB recommandés pour un usage confortable).
- **Stockage** : au moins 10 GiB libres sur le volume `/volume1` (5 GiB pour l'image Kesh + 1

GiB pour MariaDB initial + 4 GiB marge pour les imports bancaires, exports PDF/ZIP, et croissance données).

- **Paquet Container Manager** installé via Package Center (gratuit, fourni par Synology). Pour DSM 7.2+, c'est l'unique paquet officiel — il remplace l'ancien paquet *Docker*.

4.5.2 Étape 1 — Préparer la structure de répertoires

Via SSH (recommandé) ou File Station (alternative GUI), créer la structure suivante sous `/volume1/docker/kesh/` :

```
# Connexion SSH au NAS (activer SSH dans Panneau de configuration > Terminal
  et SNMP)
ssh admin@<IP-NAS>

# Création des dossiers
sudo mkdir -p /volume1/docker/kesh
cd /volume1/docker/kesh

# Permissions : utilisateur DSM courant + group docker (uid/gid varient
  selon DSM)
# Container Manager rendra les volumes accessibles automatiquement
sudo chown -R $(id -u):$(id -g) /volume1/docker/kesh
```

4.5.3 Étape 2 — Récupérer docker-compose.yml et .env

Télécharger le [docker-compose.yml](#) canonique depuis le dépôt Kesh et créer le fichier `.env` à partir du template (voir section 5.1 pour la liste complète) :

```
cd /volume1/docker/kesh
curl -fsSL
  https://raw.githubusercontent.com/guycorbaz/kesh/main/docker-compose.prod.yml
  -o docker-compose.yml
curl -fsSL
  https://raw.githubusercontent.com/guycorbaz/kesh/main/.env.example -o .env

# Secrets obligatoires : KESH_JWT_SECRET, DATABASE_URL (et les mots de
# passe de votre MariaDB) -- ils DOIVENT remplacer les placeholders de .env
openssl rand -hex 32 # → KESH_JWT_SECRET
openssl rand -hex 32 # → mot de passe MariaDB (repris dans DATABASE_URL)
# hexadécimal : un « / » de base64 casserait DATABASE_URL
# Optionnel : sans KESH_ADMIN_PASSWORD, l'administrateur se crée à l'écran
# /setup ; si vous décommentez la ligne, remplacez son placeholder
openssl rand -base64 24 # → KESH_ADMIN_PASSWORD

# Éditer .env avec vi/nano pour remplacer les <GENERATE_ME: ...> et <EDIT:
  ...>
sudo nano .env
```

`docker-compose.prod.yml` rattache `kesh-api` à un réseau Docker **externe**, `frontend`, qu'il ne crée pas : créez-le une fois, avant le premier démarrage (`sudo docker network create frontend`). C'est aussi par lui que les conteneurs de sauvegarde atteignent la base (§7.8).

! Attention

Sécurité : ne jamais laisser les valeurs par défaut `changeme` ou `change-me-32-bytes-...`, le placeholder `<GENERATE_ME: ...>` du gabarit, ni aucune valeur entre chevrons `<...>` dans `.env` pour `KESH_JWT_SECRET` et `KESH_ADMIN_PASSWORD`. Kesh fait fail-fast au boot si ces placeholders restent — c'est intentionnel pour protéger les utilisateurs débutants qui oublient cette étape.

4.5.4 Étape 3 — Importer le projet dans Container Manager

Méthode GUI (recommandée pour débutants)

1. Ouvrir **Container Manager** depuis le menu DSM.
2. Aller dans l'onglet **Projet** (panneau gauche).
3. Cliquer **Créer**.
4. Renseigner :
 - **Nom du projet** : `kesh`
 - **Chemin** : naviguer vers `/volume1/docker/kesh`
 - **Source** : *Utiliser docker-compose.yml existant*
5. Container Manager affiche un preview du compose. Vérifier que le service `kesh-api` apparaît (et `traefik` si vous l'avez ajouté). La base n'est pas dans ce compose : c'est la vôtre.
6. Cliquer **Suivant** → **Démarrer le projet maintenant** → **Terminé**.

i Note

Pour appliquer ensuite une modification de `.env` ou du compose, passez par SSH : `sudo docker compose up -d` dans `/volume1/docker/kesh` (méthode CLI ci-dessous). L'action de l'interface de Container Manager qui recréerait le conteneur et relirait `.env` n'est pas documentée ici.

```
cd /volume1/docker/kesh

# Container Manager DSM expose `docker compose` (plugin v2)
sudo docker compose pull
sudo docker compose up -d

# Vérifier l'état
sudo docker compose ps
sudo docker compose logs -f kesh-api # Ctrl+C pour quitter
```

4.5.5 Étape 4 — Configurer le reverse proxy DSM (Application Portal)

Méthode CLI (équivalente, scriptable) DSM intègre nativement un reverse proxy HTTPS via le *Portail des applications*. Cette option est plus simple que Nginx/Caddy/Traefik pour les petites installations qui n'exposent pas Kesh à Internet (LAN-only).

1. Ouvrir **Panneau de configuration** → **Portail de connexion** → onglet **Avancé** → bouton **Proxy inversé**.
2. Cliquer **Créer**.
3. Renseigner :
 - **Nom de la description** : `Kesh comptabilité`
 - **Source** : Protocole `HTTPS` , Hostname `kesh.local` (ou votre domaine), Port `443`
 - **Destination** : Protocole `HTTP` , Hostname `localhost` , Port `80`
4. Onglet **En-têtes personnalisés** → ajouter :
 - `X-Forwarded-Proto` : `https`
 - `X-Real-IP` : `$remote_addr`
5. Onglet **Avancé** → cocher `HTTP/2` et `HSTS` (Strict-Transport-Security).
6. Cliquer **Enregistrer**.

Certificat HTTPS via DSM Let's Encrypt DSM intègre un client ACME pour Let's Encrypt :

1. **Panneau de configuration** → **Portail de connexion** → onglet **Certificat**.
2. Cliquer **Ajouter** → **Obtenir un certificat de Let's Encrypt**.
3. Renseigner le domaine (`kesh.exemple.ch`), l'email, et un *Subject Alternative Name* optionnel.
4. DSM valide le challenge HTTP-01 ou DNS-01 selon configuration, et installe le certificat automatiquement.
5. Lier le certificat au hostname dans **Paramètres** → sélectionner le service `kesh.exemple.ch` et choisir le nouveau certificat.

i Note

Pour les déploiements exposés sur Internet avec besoin de firewall applicatif (rate limiting, WAF), préférer **Traefik en sidecar** (section 4.4) au Portail DSM. Le Portail couvre les besoins LAN-only ou les déploiements simples derrière un VPN/WireGuard.

4.5.6 Étape 5 — Vérification post-installation

Tester l'instance via le hostname configuré :

```
# Healthcheck (doit retourner JSON avec db:true)
curl -k https://kesh.local/health

# Réponse attendue (Story 10-3) :
# {"status":"ok", "db":true, "version":"<version
  dynamique>", "forgotPasswordEnabled":false}
```

Ouvrir un navigateur sur <https://kesh.local> : sur une base vide sans `KESH_ADMIN_*`, l'écran `/setup` crée le compte administrateur ; sinon, se connecter avec le compte admin renseigné dans `.env`. Le wizard d'onboarding démarre automatiquement (création de la première company, choix du plan comptable, configuration des exercices).

4.5.7 Limitations Container Manager DSM

- **Auto-restart** : Container Manager redémarre automatiquement les containers au reboot DSM (équivalent `restart: unless-stopped` dans compose). Pas de configuration supplémentaire requise.
- **Logs** : accessibles via Container Manager GUI (onglet *Container* → sélectionner `kesh-api` → *Détails* → *Journaux*) ou via SSH `docker compose logs -f`.
- **Mises à jour Kesh** : nécessitent un `docker compose pull && docker compose up -d` via SSH (Container Manager GUI ne propose pas de rebuild automatique sur push d'image). Voir section 8 pour la procédure complète.
- **Backup et restore** : voir section dédiée 7.8 pour les méthodes natives DSM (Hyper Backup, Snapshot Replication).

5 Configuration

5.1 Variables d'environnement complètes

Toute la configuration Kesh passe par des variables d'environnement, conformément à la philosophie [Twelve-Factor App](#).

Comment `.env` atteint Kesh. Docker Compose lit `.env` pour remplir les `${...}` du fichier compose ; il ne le transmet pas au conteneur. **Seules les variables listées sous `environment` du service `kesh-api` atteignent Kesh.** Les compose fournis les listent toutes, sauf `KESH_STATIC_DIR` et `KESH_LOCALES_DIR` (fixées par l'image Docker) et `KESH_TEST_MODE` (jamais en production). Une modification de `.env` s'applique par `docker compose up -d`, **pas**

par `docker compose restart`, qui redémarre le conteneur avec son environnement d'origine. Avec les compose fournis, une variable facultative que `.env` ne pose pas, ou laisse vide, prend son défaut, sans avertissement. Plus généralement, une ligne laissée vide vaut une ligne absente, sauf, sous Docker, `KESH_LOG_FILE_PATH` (vide = pas de journal fichier, absente = le défaut du compose; hors Docker, vide et absente désactivent toutes deux le journal fichier); les espaces de tête et de fin sont ignorés.

Une valeur qui contient `$` se met entre apostrophes simples (`KESH_SMTP_PASSWORD='pa$word'`), notamment pour `KESH_SMTP_PASSWORD` — sinon Compose la tronque au `$`, avec un seul avertissement à `docker compose up`. L'écriture `$$` n'échappe le `$` que pour Compose (un `cargo run` hors Docker lit `pa$$word` comme `pa`). Attention : `docker compose config` réaffiche tout `$` en `$$` — ce n'est pas la valeur reçue par Kesh.

Les tableaux ci-dessous listent **toutes** les variables lues par `kesh-api` (référence : `crates/kesh-api/src/config.rs` et `main.rs`), par thème. Une valeur hors borne ou invalide déclenche un warning au démarrage et retombe sur le défaut; les variables « obligatoires » font échouer le démarrage si absentes ou vides.

Obligatoires (démarrage refusé si absentes ou vides).

Variable	Description
<code>DATABASE_URL</code>	URL de connexion MariaDB : <code>mysql://USER:PASS@HÔTE:PORT/BASE</code> .
<code>KESH_JWT_SECRET</code>	Secret HS256 des JWT. $\geq$ 32 octets . Générer : <code>openssl rand -hex 32</code> . Les espaces de tête et de fin ne font pas partie du secret et ne comptent pas dans les 32 octets. Rejeté s'il contient <code>change-me</code> ou <code>GENERATE_ME</code> (sans égard à la casse), ou s'il est un gabarit entre chevrons <code><...></code> .

Note — `MARIADB_ROOT_PASSWORD`, `MARIADB_USER`, `MARIADB_PASSWORD`, `MARIADB_DATABASE` ne sont pas lues par `kesh-api` : elles configurent le **conteneur MariaDB** de `docker-compose.yml`, qui en compose `DATABASE_URL`. Les deux mots de passe y sont **obligatoires** (sans eux, Compose refuse de démarrer) et ne sont lus par MariaDB qu'à la **création** de la base — pour les changer : §8.3.2. `docker-compose.prod.yml` ne les lit pas : `DATABASE_URL` y vient de `.env` (mot de passe hexadécimal).

Serveur & réseau.

Variable	Description	Défaut
KESH_HOST	Adresse d'écoute (0.0.0.0 en Docker, reverse proxy en frontal).	127.0.0.1
KESH_PORT	Port d'écoute (0/invalid → 80).	80
KESH_STATIC_DIR	Répertoire du SPA compilé servi en fallback. Fixé par l'image Docker (/app/static); frontend/build hors Docker.	frontend/build
KESH_LOCALES_DIR	Répertoire des traductions (FTL). Fixé par l'image Docker (/app/locales); dossier du dépôt hors Docker.	embarqué
KESH_LANG	Locale de l'instance (fr/de/it/en).	fr

Compte administrateur & onboarding.

Variable	Description	Défaut
KESH_ADMIN_USERNAME	Admin bootstrap (base vide) <i>ou</i> recovery break-glass. Vide → onboarding /setup.	<i>vide</i>
KESH_ADMIN_PASSWORD	Mot de passe admin. Si renseigné : ≥ 12 caractères, refusé s'il vaut <code>changeme</code> , contient <code>GENERATE_ME</code> ou est un gabarit entre chevrons <code><...></code> .	<i>vide</i>
KESH_PRODUCTION_RESET	Autorise le reset d'onboarding au-delà de l'étape 2 (<code>true/1</code>).	<code>false</code>

Authentification, session & sécurité.

Variable	Description	Défaut
<code>KESH_JWT_EXPIRY_MINUTES</code>	Durée access token. Borne [1, 1440].	15
<code>KESH_REFRESH_TOKEN_MAX_LIFETIME_DAYS</code>	Durée absolue refresh token. Borne [1, 365].	30
<code>KESH_REFRESH_INACTIVITY_MINUTES</code>	Inactivité avant expiration refresh (sliding). Borne [1, 1440].	15
<code>KESH_RATE_LIMIT_WINDOW_MINUTES</code>	Fenêtre de comptage des échecs login. Borne [1, 1440].	15
<code>KESH_RATE_LIMIT_MAX_ATTEMPTS</code>	Échecs login par IP avant blocage. Borne [1, 100].	5
<code>KESH_RATE_LIMIT_BLOCK_MINUTES</code>	Durée de blocage d'une IP. Borne [1, 1440].	30
<code>KESH_PASSWORD_MIN_LENGTH</code>	Longueur min. des mots de passe. Borne [8, 128].	12
<code>KESH_COOKIE_SECURE</code>	Flag Secure des cookies (true/false). false unique-ment en LAN HTTP strict.	true

Journalisation.

Variable	Description	Défaut
<code>RUST_LOG</code>	Niveau/directives (trace..error). Baseline <code>sqlx=warn</code> maintenue en debug (anti-fuite SQL).	info
<code>KESH_LOG_FILE_PATH</code>	Fichier de log (rotation). Vide → stdout seul.	§5.2
<code>KESH_LOG_FILE_ROTATION</code>	daily/hourly/never.	daily
<code>KESH_LOG_FILE_MAX_FILES</code>	Fichiers de log conservés.	7
<code>KESH_LOG_FILE_FORMAT</code>	pretty/json.	pretty

Import bancaire, export & sauvegarde.

Variable	Description	Défaut
<code>KESH_BANK_IMPORT_MAX_MB</code>	Taille max fichier bank-import (CAMT.053/CSV), MiB. Borne [1, 100].	10
<code>KESH_ADMIN_EXPORT_INMEM_MB</code>	Plafond in-memory de l'export .keshbackup (au-delà : streaming). Borne [1, 2048].	50
<code>KESH_ADMIN_IMPORT_MAX_MB</code>	Taille max upload .keshbackup à l'import. Borne [1, 10240].	512
<code>KESH_ADMIN_BACKUP_DIR</code>	Dossier de sauvegarde de sécurité pré-import, monté sur ./backup par les compose fournis. Hors Docker : un dossier inscriptible. Vide = défaut.	/data/backup

Import de factures depuis un dossier (§5.1.1).

Variable	Description	Défaut
<code>KESH_DOCUMENTS_DIR</code>	Archivage des justificatifs importés et des PDF de factures émises ({sha256}.{ext}, hors DB) — à sauvegarder .	/data/documents
<code>KESH_INBOX_DIR</code>	Dossier inbox scruté à l'import.	/data/inbox
<code>KESH_INBOX_MAX_FILE_BYTES</code>	Taille max d'un fichier inbox (octets). Borne [1, 524288000].	26214400
<code>KESH_INBOX_MAX_FILES_PER_RUN</code>	Fichiers par déclenchement d'import. Borne [1, 10000].	200
<code>KESH_INBOX_MAX_PDF_PAGES</code>	Pages PDF rendues par fichier (anti-DoS). Borne [1, 500].	20

E-mails sortants (SMTP) : envoi de factures et recovery de mot de passe.

La configuration SMTP sert **deux fonctions indépendantes** (v0.6) : l'**envoi de factures par e-mail** s'active dès que les quatre variables `KESH_SMTP_HOST/USER/PASSWORD/FROM` sont renseignées (sans elles, le bouton d'envoi est simplement grisé — jamais de refus de démarrage) ; le **recovery de mot de passe** exige *en plus* `KESH_FEATURE_FORGOT_PASSWORD=true` et `KESH_PUBLIC_BASE_URL` — et là, si le flag est actif, la config complète est **obligatoire** (dé-

mariage refusé sinon).

Variable	Description	Défaut
<code>KESH_FEATURE_FORGOT_PASSWORD</code>	Active le recovery par email (sinon : break-glass via <code>KESH_ADMIN_*</code>).	<code>false</code>
<code>KESH_SMTP_HOST</code>	Hôte SMTP (sans port).	<i>vide</i>
<code>KESH_SMTP_PORT</code>	Port SMTP (587 = STARTTLS). Borne [1, 65535].	587
<code>KESH_SMTP_USER</code>	Utilisateur SMTP.	<i>vide</i>
<code>KESH_SMTP_PASSWORD</code>	Secret SMTP (masqué dans les logs).	<i>vide</i>
<code>KESH_SMTP_FROM</code>	Adresse expéditrice (email valide).	<i>vide</i>
<code>KESH_SMTP_TLS</code>	STARTTLS sur la connexion.	<code>true</code>
<code>KESH_PUBLIC_BASE_URL</code>	Base publique du lien de reset.	<i>vide</i>

Développement / test (jamais en production).

Variable	Description	Défaut
<code>KESH_TEST_MODE</code>	Expose <code>/api/v1/_test/*</code> (reset+seed). <code>true/1</code> + bind loop-back obligatoire.	<code>false</code>

Chemins host des bind mounts (`docker-compose`, pas lus par l'app).

Où les dossiers du conteneur sont montés sur l'hôte, pour un accès depuis le PC (SMB / Synology Drive). Ce tableau vaut pour `docker-compose.yml` **seul** : pointez alors vers un dossier partagé DSM (`/volume1/...`). Avec `docker-compose.prod.yml` (installation Synology de ce manuel), les montages sont fixes — `./inbox`, `./documents`, `./log` et `./backup` dans le répertoire du compose — et ces trois variables sont **sans effet** (une version configurable est suivie par l'issue #558). Les deux compose montent en outre `./backup` sur `/data/backup` (sauvegarde de sécurité pré-import, §7.9), **sans** variable d'hôte : ce montage n'a pas de ligne dans le tableau.

Variable	Monte sur	Défaut host
<code>KESH_INBOX_HOST_DIR</code>	<code>/data/inbox</code>	<code>./inbox</code>
<code>KESH_DOCUMENTS_HOST_DIR</code>	<code>/data/documents</code>	<code>./documents</code>
<code>KESH_LOG_HOST_DIR</code>	<code>/var/log/kesh</code>	<code>./log</code>

5.1.1 Import de factures depuis un dossier

Kesh peut importer des factures fournisseurs (PDF ou images porteurs d'un Swiss QR Code) déposées dans un dossier, décoder le QR côté serveur et créer des factures « à compléter ». Deux dossiers sont à configurer, montés en volumes persistants (cf. `docker-compose`) :

- `KESH_INBOX_DIR` (défaut `/data/inbox`) — le dossier où déposer les fichiers de factures. Sur un NAS Synology, montez-y un **dossier partagé** accessible via File Station (par ex. un bind mount `./inbox:/data/inbox`) pour pouvoir y glisser vos factures. À l'import réussi, le fichier d'origine est **supprimé** de l'inbox (la copie archivée fait foi) ; les fichiers en échec sont déplacés dans le sous-dossier `failed/`.
- `KESH_DOCUMENTS_DIR` (défaut `/data/documents`) — le dossier d'archivage des justificatifs (copies des fichiers importés, nommées `{sha256}.{ext}`) **et des PDF de factures émises** : depuis la v0.12.1, le PDF d'une facture est figé au premier téléchargement ou envoi et conservé ici (`{sha256}.pdf`). Ce sont des **pièces à conserver**, que la base ne contient pas : ce dossier **doit être sauvegardé avec la base** (cf. §7.3). **Ne jamais pointer** ces deux dossiers vers `/tmp` : leur contenu serait perdu au redémarrage du conteneur.

Note Synology (chemins symboliques). Sur DSM, un dossier partagé comme `/data/inbox` est souvent un lien symbolique vers `/volume1/.../inbox`. Kesh canonicalise automatiquement les chemins au démarrage pour que le contrôle anti-traversal fonctionne — aucune action requise.

Réglage `KESH_INBOX_MAX_PDF_PAGES`. Le décodage des PDF rend chaque page en image (via la bibliothèque native `pdfium`) jusqu'à trouver le QR. Augmentez cette borne si vos factures dépassent 20 pages ; diminuez-la pour limiter la charge sur des PDF volumineux.

Limitations connues.

- La bibliothèque `pdfium` n'est pas isolée : un PDF gravement malformé peut faire planter le processus serveur (le conteneur redémarre alors automatiquement). Limitez l'inbox aux fichiers de confiance.
- L'inbox est un dossier **unique** par instance.

5.1.2 Monitoring

Kesh n'expose pas encore d'endpoint de métriques Prometheus ni d'intégration Sentry en v0.x. Le monitoring se fait via le **healthcheck** `/health` (état DB + version, utilisé par le `healthcheck` Docker) et les **logs** (§5.2).

5.2 Configuration des logs fichier

Depuis la version 0.1.1, Kesh peut écrire ses logs dans un fichier avec rotation native, **en plus** de la sortie standard consultable via `docker logs kesh-api`. C'est utile car la rétention

de `docker logs` est purgée à chaque recréation du conteneur, alors qu'un fichier persiste et peut être inclus dans le backup.

En production (`docker-compose.prod.yml`), cette sortie est **activée par défaut** : les logs sont écrits dans `/var/log/kesh/kesh.log` à l'intérieur du conteneur, monté sur le répertoire `./log/` de l'hôte (à côté du `.env` et de la base, donc couvert par le même backup).

Variable	Description	Défaut
<code>KESH_LOG_FILE_PATH</code>	Chemin complet du fichier de log. Vide = logs fichier désactivés (sortie standard uniquement).	<code>/var/log/kesh/kesh.log</code> (en prod)
<code>KESH_LOG_FILE_ROTATION</code>	Stratégie de rotation : <code>daily</code> , <code>hourly</code> ou <code>never</code> .	<code>daily</code>
<code>KESH_LOG_FILE_MAX_FILES</code>	Nombre de fichiers conservés. Ignoré si rotation = <code>never</code> .	<code>7</code>
<code>KESH_LOG_FILE_FORMAT</code>	Format : <code>pretty</code> (lisible) ou <code>json</code> (ingestion outillée).	<code>pretty</code>

! Attention

Le conteneur `kesh-api` s'exécute en tant que `root`, donc le répertoire `./log/` et ses fichiers appartiennent à `root:root` sur l'hôte NAS. Hyper Backup (qui tourne en `root`) les sauvegarde sans problème. Pour consulter les logs avec un compte non-`root`, utilisez `sudo tail -f ./log/kesh.log` ou ajustez les permissions du répertoire.

Pour **désactiver** les logs fichier (sortie standard uniquement), définir `KESH_LOG_FILE_PATH=` (valeur vide) dans le `.env`.

5.3 Configuration MariaDB

5.3.1 Paramètres recommandés

Modifier la configuration MariaDB pour optimiser l'usage Kesh. Créer `/etc/mysql/mariadb.conf.d/99-kesh.cnf`

```
[mariadb]
# Encodage strict pour la conformite Unicode (factures multilingues)
```

```

character-set-server = utf8mb4
collation-server     = utf8mb4_unicode_ci

# Niveau d'isolation : laisser REPEATABLE READ (default MariaDB).
# Kesh protege ses operations critiques (numeration des
# factures, rapprochement bancaire) par des verrous de ligne
# (SELECT ... FOR UPDATE) et des verrous nommes, et relance
# automatiquement une operation interrompue par un
# interblocage (jusqu'a trois tentatives).
transaction-isolation = REPEATABLE-READ

# Detection des interblocages : laisser innodb_deadlock_detect
# a sa valeur par default (ON). Desactivee, un interblocage n'est
# plus detecte : il attend innodb_lock_wait_timeout et finit en
# erreur, sans relance.

# Mode SQL strict pour eviter les coercions silencieuses
sql_mode =
    STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION,ERROR_FOR_DIVISION_BY_ZERO

# InnoDB buffer pool (~ 50% de la RAM disponible)
innodb_buffer_pool_size = 1G
innodb_log_file_size    = 256M
innodb_flush_log_at_trx_commit = 1
innodb_flush_method     = O_DIRECT

# Logs lents pour audit performance
slow_query_log          = 1
slow_query_log_file     = /var/log/mysql/slow.log
long_query_time         = 2

# Binlog pour replication ou point-in-time recovery
log_bin                 = /var/log/mysql/mariadb-bin
binlog_format            = ROW
expire_logs_days        = 14

```

! Astuce

Pour une PME standard (1 société, < 50'000 écritures/an), les paramètres par défaut MariaDB suffisent largement. Cette configuration avancée n'est utile qu'à partir de très gros volumes.

5.3.2 Initialisation manuelle de la base

Si vous ne passez pas par Docker Compose, voici comment initialiser manuellement la DB :

```
mariadb -u root -p <<SQL
```

```
CREATE DATABASE kesh CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'kesh'@'%' IDENTIFIED BY '<sortie de openssl rand -hex 32>';
GRANT ALL PRIVILEGES ON kesh.* TO 'kesh'@'%;
FLUSH PRIVILEGES;
SQL
```

Au premier démarrage, l'application exécute les migrations SQLx.

5.4 Configuration multi-tenant

Le modèle de données de Kesh est multi-société : chaque enregistrement porte le `company_id` de sa société, et l'isolation est stricte (défense contre les accès croisés, ci-dessous). Mais **une installation porte une société** : l'interface ne crée pas de seconde société et ne permet pas d'en changer.

5.4.1 Concept de *company*

Une *company* représente une entité comptable (PME, association, fondation). Chaque company possède :

- Un nom commercial et une raison sociale.
- Un numéro IDE (Identifiant Unique des Entreprises suisse).
- Une langue et timezone par défaut.
- Un plan comptable propre.
- Un ou plusieurs utilisateurs (avec rôles).
- Un ou plusieurs exercices comptables.
- Ses propres factures, contacts, écritures, transactions bancaires, rapports.

5.4.2 Isolation des données

Toutes les requêtes API et toutes les requêtes SQL Kesh sont filtrées par `company_id` (cf. Story Epic 7-1 « audit multi-tenant scoping refactor »). Un utilisateur de la company A ne peut en aucun cas accéder aux données de la company B, même via une URL forgée (défense en profondeur contre IDOR).

5.4.3 Cas d'usage fiduciaire

Un fiduciaire installe **une instance par dossier client** : chaque installation porte la société de son client, et ses utilisateurs — créés par l'administrateur de l'installation — appartiennent tous à cette société. Un utilisateur créé reçoit la société de celui qui le crée ; aucun écran ne rattache un compte à une autre société.

6 Premier démarrage

6.1 Création du compte administrateur initial (onboarding self-service)

Depuis la version **v0.1.2**, Kesh adopte l'approche *self-service* commune aux applications self-hosted modernes (Jellyfin, Bitwarden, Vaultwarden) : au tout premier démarrage sur une base de données vide, un écran web `/setup` accueille l'administrateur pour créer son compte via un formulaire — sans édition préalable du fichier `.env`.

Le *bootstrap* crée seulement une **company provisoire** (placeholder) au boot. L'administrateur est ensuite créé via le formulaire `POST /api/v1/setup/admin` (route publique gated par `user_count == 0` → désactivée automatiquement en 410 Gone dès le 1^{er} admin créé).

6.1.1 Procédure (nouveau flow recommandé v0.1.2+)

1. Démarrer la stack (`docker compose up -d`) **sans** renseigner `KESH_ADMIN_*` dans `.env`.
2. Ouvrir `https://compta.exemple.ch` dans un navigateur. L'écran « **Bienvenue dans Kesh** » apparaît automatiquement.
3. Saisir un nom d'utilisateur (ex. `admin`), un mot de passe fort (≥ 12 caractères) et sa confirmation.
4. Cliquer sur « **Créer le compte administrateur** ».
5. Le wizard d'onboarding démarre automatiquement (langue, mode, coordonnées de la société, banque).

! Attention

Sécurité — bloquer l'accès réseau public avant le 1^{er} démarrage. La personne qui touche `/api/v1/setup/admin` en premier devient administrateur. Recommandé : binder loopback `127.0.0.1` (mapping Docker par défaut en prod) ou LAN privé via reverse proxy en attendant la création du compte. Le rate-limit IP (5 tentatives/15 min) limite le brute-force du formulaire.

6.1.2 Procédure alternative — bootstrap déclaratif via `.env`

Pour les déploiements automatisés (CI, Test, scripts de provisioning), il reste possible de créer l'administrateur *déclarativement* via les variables `.env` — équivalent strict du comportement v0.1.0/v0.1.1 :

1. Renseigner `KESH_ADMIN_USERNAME=admin` et `KESH_ADMIN_PASSWORD=<mot-de-passe-fort-12-chars>` dans `.env` *avant* le premier démarrage — en remplaçant la valeur entre chevrons : copiée

telle quelle, elle fait refuser le démarrage (depuis la 0.13.0).

2. Démarrer la stack (`docker compose up -d`). Le boot crée la company stub **et** l'administrateur en une seule passe.
3. Se connecter directement avec ces identifiants — pas d'écran `/setup`.
4. **Retirer les variables `KESH_ADMIN_*` de `.env`** après la première connexion réussie, puis `docker compose up -d` (sinon, chaque redémarrage logge un warning « retirer les vars » ; un `restart` garde les variables dans le conteneur).
5. Changer le mot de passe administrateur via l'UI dès que possible.

! Astuce

Le flow self-service `/setup` est préférable pour les déploiements interactifs — il évite la mauvaise pratique consistant à laisser un mot de passe en clair dans `.env`.

6.2 Récupération de mot de passe par email (SMTP)

Depuis la version **v0.2**, Kesh propose un *recovery self-service* : un utilisateur qui a oublié son mot de passe demande un lien de réinitialisation envoyé par email (lien valable **30 minutes**, à usage unique). La fonctionnalité est **désactivée par défaut** (opt-in) : sans configuration SMTP, le seul recours reste le break-glass administrateur (section suivante).

i Note

Depuis la v0.6, la configuration SMTP décrite ici sert **aussi** à l'envoi de factures par email (§6.3) — les deux fonctions sont indépendantes : configurer le SMTP *sans* activer `KESH_FEATURE_FORGOT_PASSWORD` active l'envoi de factures seul.

6.2.1 Variables de configuration

Variable	Description	Défaut
<code>KESH_FEATURE_FORGOT_PASSWORD</code>	Active le recovery par email.	<code>false</code>
<code>KESH_SMTP_HOST</code>	Hôte du serveur SMTP.	—
<code>KESH_SMTP_PORT</code>	Port SMTP.	587
<code>KESH_SMTP_USER</code>	Identifiant SMTP.	—
<code>KESH_SMTP_PASSWORD</code>	Mot de passe SMTP (secret — jamais log-gé).	—
<code>KESH_SMTP_FROM</code>	Adresse expéditrice des emails.	—
<code>KESH_SMTP_TLS</code>	STARTTLS sur la connexion SMTP.	<code>true</code>
<code>KESH_PUBLIC_BASE_URL</code>	URL publique de l'instance, utilisée pour construire le lien de réinitialisation (sans slash final).	—

Fail-fast au démarrage : si `KESH_FEATURE_FORGOT_PASSWORD=true` mais que l'une des variables `KESH_SMTP_HOST`, `KESH_SMTP_USER`, `KESH_SMTP_PASSWORD`, `KESH_SMTP_FROM` ou `KESH_PUBLIC_BASE_URL` est absente ou vide, le serveur **refuse de démarrer** avec un message listant les variables manquantes — plutôt qu'un recovery silencieusement cassé.

! Attention

Laisser `KESH_SMTP_TLS=true` (défaut). Le lien de réinitialisation contient un jeton secret : sans TLS, il transite **en clair** entre Kesh et votre serveur SMTP. Ne désactiver qu'en toute connaissance de cause (relais local de confiance).

6.2.2 Exemples de providers

```
# Gmail (mot de passe d'application requis, pas le mot de passe du compte)
KESH_SMTP_HOST=smtp.gmail.com
KESH_SMTP_PORT=587
KESH_SMTP_USER=mon-compte@gmail.com
KESH_SMTP_PASSWORD=<mot-de-passe-application-16-chars>
KESH_SMTP_FROM=mon-compte@gmail.com

# Postfix local (relais LAN) -- les 4 vars USER/PASSWORD/FROM/PUBLIC_BASE_URL
# restent OBLIGATOIRES (fail-fast), même si le relais n'exige pas d'auth :
# renseigner les identifiants du relais ou des valeurs convenues avec lui.
KESH_SMTP_HOST=192.168.1.10
KESH_SMTP_PORT=25
```

```
KESH_SMTP_TLS=false # uniquement si le relais est de confiance
KESH_SMTP_USER=kesh
KESH_SMTP_PASSWORD=<mot-de-passe-du-relais>
KESH_SMTP_FROM=kesh@mondomaine.ch
KESH_PUBLIC_BASE_URL=https://kesh.mondomaine.ch

# Synology Mail Server (NAS) -- avec un compte dédié du Mail Server.
KESH_SMTP_HOST=nas.local
KESH_SMTP_PORT=587
KESH_SMTP_USER=kesh@nas.local
KESH_SMTP_PASSWORD=<mot-de-passe-du-compte-dedie>
KESH_SMTP_FROM=kesh@nas.local
KESH_PUBLIC_BASE_URL=https://kesh.mondomaine.ch
```

6.2.3 Fonctionnement et garanties

- **Anti-énumération** : la page « Mot de passe oublié ? » répond toujours par le même message générique, que le compte existe ou non. Un attaquant ne peut pas tester l'existence d'un compte.
- **Usage unique + expiration** : le lien expire après 30 minutes et ne fonctionne qu'une fois. Seule une empreinte du jeton est stockée en base.
- **Révocation des sessions** : après réinitialisation, toutes les sessions actives du compte sont déconnectées.
- **Comptes sans email** : un utilisateur sans adresse email n'est pas récupérable par ce flux (le formulaire répond quand même par le message générique). Renseignez les emails via la page *Utilisateurs* — ou utilisez le break-glass (section suivante).
- **Limitation de débit** : 5 requêtes de recovery par IP et par fenêtre de 15 minutes (blocage 30 minutes au-delà). Chaque demande est tracée dans le journal d'audit (`auth.password_reset_requested` / `auth.password_reset_completed`).

6.2.4 Test après configuration (recommandé)

La CI ne peut pas vérifier la délivrabilité réelle des emails. Après avoir activé la fonctionnalité, effectuez un test manuel de bout en bout :

1. Renseignez votre adresse email sur votre compte (page *Utilisateurs*).
2. Déconnectez-vous, cliquez *Mot de passe oublié ?*, saisissez votre nom d'utilisateur ou votre email.
3. Vérifiez la réception de l'email (y compris le dossier indésirables) et suivez le lien pour valider le flux complet.
4. En cas d'échec : les erreurs d'envoi SMTP sont visibles dans les logs serveur (`docker logs kesh-api` , chercher `envoi email échoué`) — elles ne sont jamais exposées côté navigateur (anti-énumération).

Si votre instance envoie vers plusieurs domaines de messagerie (Gmail, Infomaniak, serveur d'entreprise...), testez au moins un destinataire par domaine : la délivrabilité (SPF/DKIM du domaine expéditeur) varie selon le provider.

6.2.5 Limitations connues

- **Horloges synchronisées requises** : la validité du lien (30 min) compare l'horloge de l'application à celle de MariaDB. Un décalage (NTP absent, timezone MariaDB non-UTC) raccourcit ou allonge la fenêtre — un offset d'une heure rendrait les liens inutilisables. En pratique : activez NTP sur l'hôte ; avec le `docker-compose` fourni, les deux conteneurs partagent l'horloge de l'hôte.
- **Reverse proxy et limitation de débit** : la limite « 5 requêtes/15 min » est comptée **par IP source TCP**. Derrière un reverse proxy, tous les clients partagent l'IP du proxy : 5 demandes (même légitimes, d'utilisateurs différents) bloquent le recovery 30 minutes pour toute l'installation. Suivi : [issue #173](#) (support `X-Forwarded-For` opt-in).
- **Blocage = durée du lien** : un utilisateur qui épuise ses 5 essais à mi-parcours doit attendre 30 minutes — et son lien aura expiré entre-temps.
- **Arrêt du serveur pendant l'envoi** : l'email part en tâche de fond après la réponse ; un redémarrage du conteneur à cet instant précis peut perdre l'envoi. La demande reste tracée dans le journal d'audit (`auth.password_reset_requested` sans `auth.password_reset_completed` correspondant), mais l'échec d'envoi lui-même n'apparaît pas dans les logs (le processus s'est arrêté avant de pouvoir l'écrire — un échec SMTP *ordinaire*, lui, est toujours loggé). Il suffit de refaire une demande.

! Astuce

Le recovery par email et le break-glass (section suivante) sont **complémentaires** : le premier couvre les utilisateurs au quotidien sans intervention de l'admin ; le second reste la solution de dernier recours (admin lui-même, SMTP en panne, compte sans email).

6.3 Envoi de factures par e-mail

Depuis la version **v0.6**, Kesh envoie les QR-factures PDF directement par e-mail depuis la fiche facture (bouton *Envoyer par e-mail*, réservé aux rôles Comptable et Administrateur). Le message est pré-rempli dans la **langue du contact** avec une civilité personnalisée, et le destinataire est **verrouillé** sur l'adresse e-mail de la fiche contact (sécurité anti-exfiltration).

6.3.1 Activation

L'envoi s'active dès que les quatre variables `KESH_SMTP_HOST`, `KESH_SMTP_USER`, `KESH_SMTP_PASSWORD` et `KESH_SMTP_FROM` sont renseignées (§6.2 pour les exemples de providers). **Dégradation gracieuse** : sans configuration SMTP, le serveur démarre normalement et le bouton d'envoi est simplement grisé avec une explication — jamais de refus de démarrage pour cette fonction.

L'état est exposé par le healthcheck : `GET /health` renvoie `smtpConfigured: true|false`.

6.3.2 Identité d'expédition

- **From** : toujours l'adresse `KESH_SMTP_FROM`, complétée du **nom de votre société** en nom d'affichage (jamais une adresse fournie par l'utilisateur).
- **Reply-To** : l'e-mail de contact de la société s'il est renseigné (*Paramètres* → *Organisation* → *E-mail (adresse de réponse)*), réservé à l'administrateur — les réponses de vos clients arrivent alors dans votre boîte habituelle.

! Attention

Délivrabilité (SPF/DKIM) : utilisez le SMTP de votre fournisseur de messagerie, dont le domaine expéditeur est déjà authentifié (SPF/DKIM) ; n'usurpez pas un domaine tiers. Une adresse `KESH_SMTP_FROM` incohérente avec le domaine du relais finit en spam — ou est rejetée — chez vos clients.

6.3.3 Garanties et limites

- **Anti-abus** : 20 envois par fenêtre de 15 minutes et par utilisateur (puis blocage temporaire 15 minutes).
- **Traçabilité** : chaque envoi est journalisé dans le journal d'audit (`invoice.emailed`, avec destinataire et objet).
- **Renvoi** : autorisé (chaque renvoi écrase la date d'envoi affichée et laisse sa propre entrée au journal d'audit).

i Note

« Envoyée » signifie **remise au serveur SMTP**, pas accusé de réception : un rebond ultérieur (adresse invalide, boîte pleine) n'est pas remonté dans Kesh. En cas d'échec de remise immédiat (serveur SMTP injoignable), la facture n'est **pas** marquée envoyée et un message d'erreur clair est affiché.

6.4 Modèles d'e-mail

Le contenu des e-mails d'envoi de facture se personnalise dans *Paramètres* → *Modèles d'e-mail* (réservé à l'administrateur) : objet et corps, séparément pour chacune des quatre langues (FR/DE/IT/EN — la langue utilisée à l'envoi est celle du contact, sinon celle de l'instance).

- **Variables** : `{salutation}`, `{contactName}`, `{invoiceNumber}`, `{amount}`, `{dueDate}`, `{companyName}` — remplacées automatiquement à l'envoi (montants et dates au format suisse). Une variable inconnue est refusée à l'enregistrement avec un message explicite.
- **Variables propres aux rappels** : `{reminderLevel}` (niveau), `{reminderFee}` (frais du seul niveau envoyé), `{totalDue}` (**reste dû** de la facture **plus** les frais de rappel cumu-

lés), `{daysOverdue}` (jours de retard) et `{feeNotice}` (phrase disant les frais compris dans `{totalDue}`, précédée d'une espace, **vide** sans frais); `{amount}` reste le montant total de la facture. Les modèles fournis n'écrivent aucune phrase de frais en dur et passent par `{feeNotice}` : à frais nuls, ils n'en parlent pas. Un modèle **personnalisé** qui écrit `{reminderFee}` en toutes lettres affichera « 0.00 » sans frais — c'est à vous de l'éviter.

- **Zéro configuration requise** : un modèle par défaut soigné est fourni pour chaque langue — une PME peut envoyer une facture correcte sans jamais ouvrir cette section.
- **Restaurer le défaut** : supprime la personnalisation d'une langue et revient au modèle fourni (action confirmée par une modale).

6.5 Rappels débiteurs (réglages)

Depuis la version **v0.7**, Kesh gère un cycle de **rappels de paiement** (relances débiteurs) multi-niveaux. Les réglages se trouvent dans *Paramètres* → *Rappels débiteurs* (réservé à l'administrateur). L'écran de relance côté utilisateur est décrit dans le manuel utilisateur « Relancer les débiteurs ».

6.5.1 Niveaux de rappel

Chaque **niveau** définit un **délai** (en jours après l'échéance de la facture) et des **frais de rappel** (montant fixe en CHF). Trois niveaux par défaut sont fournis d'origine (*seed* automatique au premier accès) : 1er rappel, 2e rappel, dernier rappel avant poursuite. On peut ajouter, modifier ou supprimer des niveaux, et personnaliser le modèle d'e-mail de chaque niveau (ton en escalade, dans les quatre langues).

- **Délai** : le **niveau 1** devient dû $\text{échéance} + \text{grâce} + \text{délai}(1)$ (compté depuis l'échéance). Les **niveaux suivants** sont comptés depuis la **date d'envoi réelle du rappel précédent** ($\text{dernier rappel} + \text{délai}(N)$), et non depuis l'échéance : si un rappel part en retard, le suivant se décale d'autant. L'échéancier prévisionnel affiché est donc une **projection** valable si chaque rappel est envoyé à temps.
- **Frais** : bornés entre 0 et 10 000.— par niveau.
- **Modèles par niveau** : la page *Modèles d'e-mail* gère plusieurs types (envoi de facture, rappel); pour les rappels, un sélecteur de niveau permet de personnaliser objet et corps. La résolution se fait en cascade : modèle du niveau N → modèle générique → défaut fourni.

6.5.2 Période de grâce

La **période de grâce** (en jours) diffère le premier rappel après l'échéance, pour absorber les délais postaux et bancaires. Un **échancier prévisionnel** affiché en direct montre à quelle date chaque niveau serait proposé (« 1er rappel proposé 15 j après l'échéance, 2e à 25 j... »).

! Attention

Les **frais de rappel** exigent une **base contractuelle** : ils ne sont légalement exigibles que si vos **conditions générales (CGV)** les prévoient et que le client les a acceptées. Kesh affiche un rappel à ce sujet dans l'écran de réglages.

i Note

Les frais de rappel sont **affichés mais non comptabilisés** (produit accessoire, issue #401) et **ne figurent pas sur la QR-facture** du rappel : depuis la version **0.12.1**, celle-ci demande le **reste dû** de la facture (son total TTC diminué des règlements partiels), jamais augmenté des frais. Les frais apparaissent dans le courrier de rappel, sur une ligne à part du rappel PDF et dans l'historique, à titre de créance accessoire.

6.6 J'ai oublié mon mot de passe administrateur (recovery break-glass)

Depuis la version **v0.1.2**, Kesh permet de réinitialiser le mot de passe administrateur sans accès à la base de données. Le mécanisme *recovery break-glass* réutilise les variables `KESH_ADMIN_*` dans un double-usage : sur une instance déjà configurée (au moins un user existe), renseigner ces variables avec un mot de passe différent du hash actuel déclenche un **reset du password de l'administrateur correspondant au username configuré**.

6.6.1 Procédure step-by-step

1. **Stopper le container** : `docker compose stop kesh-api` (la DB reste accessible pour les autres clients).
2. **Éditer `.env`** : décommenter (ou ajouter) les deux variables, en remplaçant la valeur entre chevrons (recopiée telle quelle, elle fait refuser le démarrage depuis la 0.13.0) :

```
KESH_ADMIN_USERNAME=admin          # le username de votre compte admin
KESH_ADMIN_PASSWORD=<nouveau-mdp-12+> # nouveau mot de passe (>= 12 chars)
```

3. **Redémarrer le container** : `docker compose up -d kesh-api`. Au boot, les logs affichent :

```
WARN bootstrap: Recovery break-glass declenche pour user 'admin'...
ERROR bootstrap: Recovery effectue -- RETIRER LES VARS DE .ENV
```

4. **Se connecter** via l'écran de login avec le nouveau mot de passe. Les sessions actives sur ce compte ont été révoquées par le recovery — les autres onglets/devices devront se reconnecter.
5. **Retirer les variables `KESH_ADMIN_*` de `.env`** (les recommenter ou les supprimer). Tant qu'elles restent en place, un `WARN bootstrap: retirer les vars de .env` apparaît à chaque redémarrage (no-op silencieux côté DB grâce à la détection « hash identique »).

6. Appliquer le retrait (`docker compose up -d kesh-api` — un `restart` garderait les variables dans le conteneur) et confirmer que les warnings bootstrap ont disparu des logs — preuve que les variables `KESH_ADMIN_*` sont bien retirées.

! Attention

Sécurité du fichier `.env`. Tant que `KESH_ADMIN_PASSWORD` est non-vidé dans `.env`, toute personne ayant accès à ce fichier peut reset le mot de passe de l'administrateur. Restreindre les permissions : `chmod 600 .env`. Le recovery génère également une entrée `admin_break_glass_reset` dans le journal d'audit (`audit_log` , conservation 10 ans Swiss CO Art. 958f) pour traçabilité.

6.6.2 Matrice de comportement au boot — vue d'ensemble

Selon l'état de la base de données et la présence des variables `KESH_ADMIN_*`, le bootstrap se comporte ainsi :

users	env set ?	match ?	Comportement
vide	non	—	Crée la company stub seule, écran <code>/setup</code> actif
vide	oui	—	Crée stub + admin déclaratif (équivalent v011-2)
existant	non	—	No-op (régime nominal post-bootstrap)
existant	oui	match + hash identique	No-op silencieux, warning «retirer les vars»
existant	oui	match + hash différent	RECOVERY : reset password + révocation tokens + <code>audit_log</code>
existant	oui	no match username	No-op + warning «no user matches»

6.7 Déploiement LAN HTTP-only (sans HTTPS) — v0.1.3+

Contexte : avant v0.1.3, Kesh émettait les cookies de session avec le flag `Secure` en mode production. Le navigateur refuse de stocker ces cookies sur une connexion HTTP non-TLS, rendant l'application inutilisable sur un déploiement LAN strict sans HTTPS (par exemple un NAS Synology privé sur un domaine RFC 8375 `*.home.arpa` derrière un reverse proxy Traefik en HTTP).

Depuis la version **v0.1.3**, la variable d'environnement `KESH_COOKIE_SECURE` permet de désactiver explicitement le flag `Secure` pour ces cas d'usage.

6.7.1 Décision : HTTPS ou HTTP-only ?

Contexte

Recommandation

Prod Internet exposée

HTTPS obligatoire (Let's Encrypt automatique via Traefik / Caddy / Synology DSM).
`KESH_COOKIE_SECURE` reste à la valeur défaut `true`.

NAS privé LAN avec domaine externe (DDNS, Cloudflare, OVH)

HTTPS recommandé (Let's Encrypt via DNS challenge).
`KESH_COOKIE_SECURE=true`.

NAS privé LAN avec domaine RFC 8375 (*.home.arpa)

2 options : (a) certificat auto-signé avec `mkcert` + Traefik file provider (browser warning à accepter une fois par poste, cookies Secure marchent) — **recommandé**.
 (b) HTTP-only avec `KESH_COOKIE_SECURE=false`, à n'utiliser que si le LAN est strictement de confiance.

Dev local pur (cargo run sans Docker)

`KESH_COOKIE_SECURE=false` pour itération rapide. Ne jamais déployer cette configuration en production.

6.7.2 Procédure HTTP-only (`KESH_COOKIE_SECURE=false`)

1. Éditer `.env` :

```
KESH_COOKIE_SECURE=false
```

Appliquer la modification : `docker compose up -d kesh-api` (un `restart` ne relit pas `.env`).

2.

3. Les logs au boot affichent un warning explicite rappelant le compromis sécurité :

```
WARN config: KESH_COOKIE_SECURE=false -- cookies session emis SANS le
flag `Secure`. Acceptable uniquement pour deploiement HTTP-only LAN
strict (e.g. domaine RFC 8375 `*.home.arpa`, NAS prive). NE JAMAIS
activer en prod exposee Internet sans HTTPS -- cookies sniffables
sur tout reseau non-TLS.
```

! Attention

Risque de sécurité avec `KESH_COOKIE_SECURE=false`. Les cookies de session sont émis sans le flag `Secure` et transitent donc en clair sur le réseau HTTP. Un attaquant ayant accès au LAN (`tcpdump`, point d'accès WiFi compromis, switch managé miroir) peut intercepter les cookies et usurper la session administrateur. À n'activer que si le LAN est strictement de confiance et physiquement isolé d'Internet. Ne jamais combiner avec une exposition Internet (port forwarding, reverse-proxy externe, etc.) sans HTTPS.

6.7.3 Valeurs acceptées et fail-fast

`KESH_COOKIE_SECURE` accepte strictement `true` / `1` (par défaut, cookies Secure activés) ou `false` / `0` (cookies sans Secure). Les espaces de tête et de fin sont ignorés ; toute autre valeur (`True`, `TRUE`, `yes`, `on` ...) refuse le démarrage avec un message d'erreur explicite. Ce parsing strict (cohérent avec `KESH_TEST_MODE`) évite l'ambiguïté qui ferait croire qu'un override est actif alors qu'il ne l'est pas.

6.8 Configuration de la company via l'onboarding

Une fois connecté, l'**assistant d'onboarding** (wizard) configure la company provisoire — il n'y a pas d'écran séparé de création de société :

1. **Langue** de l'interface, puis **mode** (guidé ou expert).
2. **Parcours** : données de démonstration (exploration) ou configuration de production.
3. **Type d'organisation** (indépendant, association, PME) et **langue comptable**.
4. **Coordonnées** de l'organisation (raison sociale, adresse, numéro IDE) — ceci renomme la company provisoire et lève la bannière « nom provisoire ».
5. **Compte bancaire** principal (optionnel, configurable plus tard).

Tant que les coordonnées ne sont pas renseignées, une bannière non-bloquante rappelle que l'entreprise porte un nom provisoire (correctif catch-22 fresh-install, v0.1.1).

6.9 Choix du plan comptable

Kesh livre trois plans comptables, dont la structure suit le plan comptable PME suisse. Le plan n'est pas choisi dans une liste : il découle du **type d'organisation** indiqué à l'onboarding.

Type d'organisation

Plan mis en place

PME	Plan comptable PME suisse, capitaux propres d'une société (capital social).
Indépendant	Même structure, adaptée à l'entreprise individuelle (capital de l'exploitant, prélèvements et apports privés).
Association	Même structure, adaptée à une association (capital de l'association, cotisations des membres, dons reçus).

Kesh n'importe pas de plan comptable : le plan mis en place s'adapte ensuite compte par compte.

! Astuce

Le plan comptable peut être modifié après création (ajout de comptes auxiliaires, sous-comptes, etc.) mais sa structure globale reste stable. Le type d'organisation se choisit une seule fois, à l'onboarding, et le plan qui en découle est mis en place à ce moment : choisir le type le plus proche de votre activité évite d'avoir à adapter le plan compte par compte ensuite.

6.10 Configuration des exercices comptables

Chaque company doit définir au moins un exercice comptable (*fiscal year*). Par défaut, l'exercice civil 1^{er} janvier – 31 décembre est créé automatiquement à la création de la company. Pour d'autres exercices (décalés, courts, longs), aller dans *Paramètres* → *Exercices*.

Clôture dans l'ordre (depuis la v0.13.0). Les exercices se clôturent **du plus ancien vers le plus récent**, symétrique de la garde de réouverture ci-dessous : la clôture d'un exercice est refusée tant qu'un exercice de date de début antérieure est encore ouvert (409 `EARLIER_FISCAL_YEAR_OPEN`, qui nomme le plus ancien à clôturer d'abord), et la création d'un exercice est refusée si un exercice postérieur est déjà clôturé (400 `LATER_FISCAL_YEAR_CLOSED`). Le bilan étant cumulatif, ces deux gardes empêchent qu'un exercice ouvert précède un exercice clôturé. Une **installation mise à jour** ou une **sauvegarde restaurée** peut porter cet état, hérité d'une version antérieure : l'import l'accepte, l'écran des exercices le signale par un bandeau, et **rien ne s'enregistre** dans l'exercice ouvert concerné tant qu'il dure — ni saisie, ni règlement, ni dévalidation, ni modification, ni suppression d'écriture (400 `LATER_FISCAL_YEAR_CLOSED`) ; il se résorbe en clôturant le plus ancien exercice ouvert, ou en rouvrant les exercices clôturés

postérieurs, du plus récent vers l'ancien (section *Exercices clôturés dans le désordre*).

Réouverture d'un exercice clôturé (Admin uniquement). La clôture d'un exercice est un verrou réversible et tracé. La **réouverture** est réservée aux **administrateurs** — plus strict que la clôture, accessible aux comptables — et n'est **pas** disponible via une clé API (opération privilégiée). Elle se fait depuis *Paramètres* → *Exercices comptables*, bouton *Réouvrir*, et exige un **motif obligatoire** conservé dans le journal d'audit (`audit_log`, action `fiscal_year.reopened`, avec l'état avant/après et l'auteur ; conservation 10 ans). Une **garde d'ordre** impose de rouvrir du plus récent vers l'ancien : si un exercice de date de début postérieure est encore clôturé, la réouverture est refusée jusqu'à ce que celui-ci soit rouvert d'abord. Un exercice rouvert redevient éditable et pourra être re-clôturé. Aucune procédure en ligne de commande n'est nécessaire ni disponible pour cette opération.

6.11 Création des utilisateurs et rôles

Kesh utilise un système de contrôle d'accès basé sur les rôles (RBAC) :

Rôle	Permissions
Admin	Toutes les opérations d'une société : gestion des utilisateurs, paramètres de facturation, taux de TVA, plan comptable, exercices, sauvegarde/restauration complète, en plus de toutes les opérations Comptable.
Comptable	Saisie et validation d'écritures, factures (clients et fournisseurs), avoirs, règlements et paiements pain.001, import bancaire, réconciliation, import de factures, rapports et exports.
Consultation	Lecture seule : consultation des écritures, factures, contacts, produits, rapports — aucune mutation, à une exception près : le premier téléchargement du PDF d'une facture le fige et l'inscrit au journal d'audit, quel que soit le rôle.

Kesh définit exactement ces **trois** rôles (`Role::Admin`, `Role::Comptable`, `Role::Consultation`). Chaque utilisateur appartient à **une seule** société (`company_id` porté par le JWT) ; il n'existe pas de rôle « super-admin » cross-société ni de sélecteur de société.

Création d'un utilisateur via l'UI (rôle Administrateur) : *menu* → *Utilisateurs* → *Nouvel utilisateur*.

7 Sauvegarde et restauration

7.1 Importance de la sauvegarde

La sauvegarde régulière de l'instance Kesh est **critique** pour deux raisons :

1. **Continuité d'activité** : perte de données = arrêt de la comptabilité PME.
2. **Conformité OLICo Art. 6** (Disponibilité) : « jusqu'à la fin du délai de conservation [10 ans], toute personne autorisée doit pouvoir, en tout temps et dans un délai raisonnable, consulter et vérifier les livres et les pièces comptables ». Une perte de données rend l'instance non-conforme.

7.2 Stratégie recommandée

- **Quotidienne** : `mariadb-dump` complet, conservé 30 jours, stocké en local + offsite (cloud).
- **Quotidienne, avec la base** : copie du répertoire des documents (`KESH_DOCUMENTS_DIR`, monté depuis `./documents` avec `docker-compose.prod.yml` ; avec `docker-compose.yml`, depuis le dossier de `KESH_DOCUMENTS_HOST_DIR` s'il est posé) — il porte les **PDF des factures émises**, que ni le dump ni l'export ne contiennent (cf. §7.3).
- **Avec le répertoire du compose** : le dossier `./backup`, où Kesh écrit la sauvegarde de sécurité prise avant chaque import d'installation, est dans le périmètre du répertoire du compose — il contient des **secrets** (§7.9).
- **Hebdomadaire** : snapshot du volume Docker `/var/lib/docker/volumes/<projet>_kesh-mariadb-data` (volume `kesh-mariadb-data` de `docker-compose.yml`, préfixé du nom du projet) (gel cohérent via `docker compose stop` puis snapshot, ou via LVM/ZFS).
- **Mensuelle** : export ZIP de souveraineté (Story 9-2b) pour chaque company depuis l'interface utilisateur.
- **Annuelle** : archivage long terme conforme OLICo Art. 9 (support non-modifiable type CD-R, ou hash signé sur support modifiable WORM).

7.3 Les PDF des factures émises

Depuis la v0.12.1, le PDF d'une facture est **figé** au premier téléchargement ou envoi : Kesh l'écrit dans `KESH_DOCUMENTS_DIR` sous le nom `{sha256}.pdf` et n'en garde en base que la référence et l'empreinte. Ce fichier est la **copie de la pièce émise** — celle qu'un contrôle demande —, et il n'est **ni dans le dump MariaDB, ni dans le fichier `.keshbackup`**¹. D'où trois règles :

- **Sauvegarder** le répertoire des documents **en même temps** que la base, et le **restaurer avec elle**.

¹L'inclusion des fichiers dans `.keshbackup` est suivie par l'issue #503.

- Une base restaurée sans ses documents répond, pour chaque facture figée, un refus 410 `INVOICE_PDF_GONE` dont le message nomme le fichier manquant. **Le remède est de restaurer ce fichier** depuis la sauvegarde du répertoire : c'est le seul moyen de retrouver l'original. Si les refus arrivent en série après une mise à jour, vérifier d'abord le **montage** du volume des documents.
- À défaut de sauvegarde, un administrateur peut **refiger** la facture depuis sa fiche (bouton *Refiger le document*, ou `POST /api/v1/invoices/{id}/pdf/refreeze`) : Kesh produit un **nouveau** document, avec les données du moment — **ce n'est pas l'original** — et inscrit au journal d'audit l'empreinte de l'ancien et du nouveau (`invoice.pdf_refrozen`). Le geste est irréversible et refusé à une clé d'API. Une facture **annulée** ne peut pas être refigée : seule la restauration du fichier la répare.

! Attention

Un fichier altéré (empreinte différente de celle enregistrée) n'est **jamais servi** : le téléchargement répond une erreur serveur et la journalise. Le refageage le refuse aussi (409 `INVOICE_PDF_INTEGRITY`) : un fichier altéré se **diagnostique**, il ne se recouvre pas. Restaurer `{empreinte}.pdf` depuis la sauvegarde ; à défaut, **écarter** le fichier altéré (le déplacer hors du répertoire), ce qui ramène au cas du fichier manquant, puis refiger.

7.4 Script de sauvegarde mariadb-dump

Voici un script bash de référence à exécuter via cron quotidien. Il lit le mot de passe root **dans le conteneur** (la commande entière passe à `sh -c` entre apostrophes) : il n'a besoin d'aucune variable posée dans la crontab. `cron` ne le lance pas depuis le répertoire du compose : `COMPOSE_DIR` le lui donne.

```
#!/bin/bash
# /usr/local/bin/kesh-backup.sh
# Sauvegarde quotidienne de la base Kesh

set -euo pipefail

COMPOSE_DIR="/opt/kesh"          # répertoire de docker-compose.yml
BACKUP_DIR="/var/backups/kesh"
RETENTION_DAYS=30
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/kesh_${TIMESTAMP}.sql.gz"

mkdir -p "${BACKUP_DIR}"
cd "${COMPOSE_DIR}"

# Dump gzip et hash SHA-256 ; mot de passe root lu dans le conteneur
docker compose exec -T mariadb sh -c \
    'mariadb-dump --single-transaction --routines --triggers \
```

```

--events --add-drop-database --databases kesh \
-u root -p"$MARIADB_ROOT_PASSWORD" ' \
| gzip > "${BACKUP_FILE}"

# Hash SHA-256 pour verification ulterieure (OLiCo Art. 9 al. 1.b ch. 1)
sha256sum "${BACKUP_FILE}" > "${BACKUP_FILE}.sha256"

# Conservation 30 jours
find "${BACKUP_DIR}" -name "kesh_*.sql.gz" -mtime +${RETENTION_DAYS} -delete
find "${BACKUP_DIR}" -name "kesh_*.sql.gz.sha256" -mtime +${RETENTION_DAYS}
-delete

echo " Sauvegarde terminee : ${BACKUP_FILE}"

```

Crontab d'exemple (sauvegarde à 2h du matin chaque jour) :

```
0 2 * * * /usr/local/bin/kesh-backup.sh >> /var/log/kesh-backup.log 2>&1
```

! Attention

Lancez le script une fois à la main après son installation, après toute mise à jour et après tout changement de mot de passe MariaDB : un échec la nuit ne s'écrit que dans `/var/log/kesh-backup.log`, que personne ne lit. C'est en particulier le seul signal d'un mot de passe root changé dans `.env` sans ALTER USER (§8.3.2).

7.5 Stockage offsite

Synchroniser les sauvegardes vers un stockage distant (S3, Backblaze B2, Wasabi, Hetzner Storage Box, OVH C14, etc.) :

```

# Exemple avec rclone
rclone sync /var/backups/kesh remote:kesh-backups \
--transfers 4 \
--checksum \
--log-file /var/log/kesh-backup-sync.log

```

7.6 Restauration depuis sauvegarde

```

# 1. Verifier l'integrite de la sauvegarde (SHA-256)
cd /var/backups/kesh
sha256sum -c kesh_20260520_020000.sql.gz.sha256

# 2. Arreter Kesh (pas la DB)
docker compose stop kesh-api

```

```
# 3. Restaurer la base (depuis le repertoire du compose, COMPOSE_DIR du
#   script de sauvegarde ; mot de passe root lu dans le conteneur)
cd /opt/kesh
zcat /var/backups/kesh/kesh_20260520_020000.sql.gz \
  | docker compose exec -T mariadb \
    sh -c 'mariadb -u root -p"$MARIADB_ROOT_PASSWORD"'

# 4. Restaurer le repertoire des documents (PDF des factures emises,
#   justificatifs) depuis SA sauvegarde, prise en meme temps que le dump
rsync -a /var/backups/kesh/documents_20260520/ ./documents/

# 5. Redemarrer Kesh
docker compose start kesh-api

# 6. Verifier le fonctionnement
curl http://localhost/health
```

7.7 Test de restauration périodique (OLICo Art. 10 al. 1)

L'article 10 al. 1 OLICo exige que « l'intégrité et la lisibilité des supports d'information sont régulièrement contrôlées ». Cela implique de **tester effectivement la restauration** au moins **annuellement**, sur un environnement de test isolé :

1. Provisionner une instance Kesh de test (machine virtuelle ou container isolé).
2. Restaurer la sauvegarde la plus récente.
3. Vérifier que l'application démarre, que les écritures sont accessibles, qu'un export ZIP de souveraineté peut être généré et que son SHA-256 est identique à celui généré sur l'instance de production avant la sauvegarde.
4. Documenter le test dans un procès-verbal écrit (date, opérateur, résultat, anomalies éventuelles). Conservation 10 ans avec les sauvegardes.

! Astuce

Automatisez ce test annuel via un workflow CI/CD : provisioning Docker éphémère, restauration, exécution de tests E2E Playwright basiques, génération du procès-verbal. Cette automatisation peut être traitée comme une story Epic 10 « Déploiement & Opérations » ou Epic ultérieur.

7.8 Backup natif sur Synology DSM (Hyper Backup + Snapshot Replication)

Pour les installations sur Synology DSM (cf. section 4.5), DSM intègre nativement deux outils de backup éprouvés : **Hyper Backup** (backup incrémental versionné, idéal cloud + LAN) et **Snapshot Replication** (snapshot Btrfs point-in-time, idéal recovery instantané). Ces outils **complètent** le dump `mariadb-dump`, ils ne le remplacent pas : ils copient le dossier du compose, `/volume1/docker/kesh`, et le dump qu'y dépose chaque nuit une **tâche planifiée** de DSM (ci-dessous) ; ils ne voient pas la base elle-même. Avec `docker-compose.prod.yml`, la base est celle que désigne `DATABASE_URL` — le paquet MariaDB 10 de DSM, un conteneur séparé ou une base gérée —, et elle n'est **pas** dans ce dossier : elle ne se sauvegarde, et ne se restaure, que par ce dump (ou là où vous l'hébergez).

i Note

Ce qui a été éprouvé, et ce qui ne l'a pas été. Les scripts de cette section (dump, rechargement, comptes, fichiers d'identifiants) ont été rejoués **hors DSM**, sur des conteneurs MariaDB 10.11 : sauvegarde, refus du compte de sauvegarde, rechargement, dump raté, empreinte fausse. Hyper Backup n'a **pas** de pré-script — constaté sur DSM —, d'où la tâche planifiée. Les **chemins de menus** de DSM cités ici (Planificateur de tâches, Hyper Backup, Snapshot Replication, pare-feu) sont à confirmer sur votre version de DSM.

7.8.1 Hyper Backup — backup incrémental + offsite intégré

Hyper Backup est l'outil principal Synology pour les backups versionnés vers stockage local externe (HDD USB, NAS distant), cloud (Synology C2 Backup, S3, Backblaze B2, Wasabi, Google Drive, OneDrive, etc.), ou serveur rsync distant. Il supporte la déduplication, le chiffrement client-side AES-256, et la rotation Smart (équivalent grandfather-father-son).

Configuration Hyper Backup pour Kesh

1. Installer **Hyper Backup** via Package Center (gratuit).
2. Lancer Hyper Backup → + **Tâche de sauvegarde de données** → choisir la destination (ex. *Synology C2 Backup* pour offsite managé, *S3 Storage* pour AWS/Wasabi/Backblaze, *Dossier local* pour HDD USB attaché au NAS).
3. Cocher les dossiers à backup :
 - **Obligatoire** : `/volume1/docker/kesh/` — le compose, `.env`, `log/`, `inbox/`, `documents/`, `backup/`, les deux fichiers d'identifiants (`kesh-dump.cnf`, `kesh-restore.cnf`), les deux scripts et le dossier `dump/`, où la tâche planifiée dépose le **dump de la base** (`kesh_pre_backup.sql.gz`). La base elle-même n'est pas dans ce dossier : seul ce dump l'y fait entrer. `documents/` porte les PDF des factures émises (§7.3) ; avec `docker-compose.yml`, c'est le dossier de `KESH_DOCUMENTS_HOST_DIR` s'il est posé — à ajouter alors à la tâche de sauvegarde. `backup/` (sauvegarde de sécurité prise avant chaque import d'installation) et `dump/` contiennent

des **secrets** — condensés des mots de passe, jetons de session — : d'où leurs droits réservés à root et le chiffrement client-side ci-dessous (§7.9). À restaurer avec la base.

4. Cocher l'application **Container Manager** dans *Applications* pour backup automatique de la configuration des projets.
5. Activer le **chiffrement client-side** (mot de passe différent du KESH_JWT_SECRET, stocké séparément offline — la perte de ce mot de passe rend les backups illisibles).
6. Configurer la rotation **Smart Recycle** : recommandé *Conservez toutes les versions des derniers 30 jours + 1 par semaine pour 12 semaines + 1 par mois pour 12 mois*. Convertit OLiCo 10 ans en pratique via archivage annuel séparé (cf. *Stockage offsite* ci-dessus).
7. Planifier la fréquence : **quotidienne à 2h du matin**, soit **après** la tâche planifiée de dump (1h30, ci-dessous). L'ordre compte : Hyper Backup copie le dump présent au moment où il passe.
8. Activer les notifications email/push DSM pour alerter d'un échec de backup.

Cohérence transactionnelle MariaDB Hyper Backup ne voit pas la base : il copie des fichiers, et la base n'est pas dans le dossier du compose. Une **tâche du Planificateur de tâches** de DSM en prend chaque nuit un dump cohérent (`--single-transaction` : une seule transaction de lecture, sans arrêter Kesh), **avant** la tâche Hyper Backup, et le dépose dans `dump/`, où Hyper Backup et les snapshots le copient avec le reste. Le dump s'écrit d'abord dans un fichier `.tmp`, renommé seulement après son succès ; un dump raté est supprimé et ne remplace pas celui de la veille. Mais Hyper Backup ignore tout de cette tâche : si elle échoue, il sauvegarde le dump de la veille **sans le signaler**. D'où la notification de la tâche et le contrôle ci-dessous.

Deux comptes, deux fichiers d'identifiants Les identifiants de la base ne sont pas lus dans `.env` : ils sont dans deux fichiers d'options MariaDB, un par usage.

- **Sauvegarde** — le **compte de sauvegarde** `kesh_backup`, créé pour elle, qui ne peut que lire (SELECT : `--single-transaction` ne verrouille aucune table). Ses identifiants vont dans `/volume1/docker/kesh/kesh-dump.cnf`, lu par le seul script de dump. Le compte Kesh conviendrait techniquement ; un fichier lu chaque nuit par une tâche planifiée porte le moindre privilège.
- **Restauration** — le **compte Kesh**, celui de `DATABASE_URL`. Le dump recrée la base (DROP DATABASE, CREATE DATABASE, puis tables et lignes) : il faut ALL PRIVILEGES sur la base, que porte le compte Kesh créé par le compose ou par l'*Initialisation manuelle de la base* (§5.3.2). Ses identifiants vont dans `/volume1/docker/kesh/kesh-restore.cnf`, lu par le seul script de rechargement (§7.8.2, *Recovery depuis Snapshot*). Le compte de sauvegarde, lui, ne peut pas recharger : sa tentative est refusée (ERROR 1044) et la base reste intacte.

Créer le compte de sauvegarde, sur votre MariaDB, avec un compte d'administration du serveur. Remplacez `kesh` par la base de `DATABASE_URL` si elle porte un autre nom (ici et dans les deux scripts, variable `BASE`), et le texte entre chevrons par un mot de passe neuf — les

chevrons ne se recopient pas :

```
CREATE USER 'kesh_backup'@'%' IDENTIFIED BY '<sortie de openssl rand -hex 32>';
GRANT SELECT ON kesh.* TO 'kesh_backup'@'%' ;
```

Chaque fichier d'identifiants a la forme suivante — `host`, `port` et `password` de la base de `DATABASE_URL`; `user` : `kesh_backup` dans `kesh-dump.cnf`, le compte Kesh dans `kesh-restore.cnf` :

```
[client]
host = "192.168.1.10"
port = 3307
user = "kesh_backup"
password = "<mot de passe en clair, décodé>"
```

Puis, en session root (`sudo -i`) :

```
cd /volume1/docker/kesh
nano kesh-dump.cnf           # compte de sauvegarde (kesh_backup)
nano kesh-restore.cnf       # compte Kesh, celui de DATABASE_URL
chown root:root kesh-dump.cnf kesh-restore.cnf
chmod 600 kesh-dump.cnf kesh-restore.cnf
mkdir -p dump avant-restauration
chmod 700 dump avant-restauration
docker pull mariadb:10.11    # une fois : le dump et le rechargement
                             l'emploient
```

- `--defaults-extra-file=...` doit être la **première** option de `mariadb-dump` comme de `mariadb` : placée après une autre, elle est refusée.
- Le mot de passe s'écrit **en clair, décodé, et entre guillemets** : celui de `DATABASE_URL` est pourcentage-encodé s'il contient `@` : `/` `%` `#` `?` (`%40` pour `@`) — décodez-le avant de l'écrire. Dans un fichier d'options, `#`, `;` et les espaces sont spéciaux hors guillemets ; **même entre guillemets**, un `"` s'écrit `\` et un `\` s'écrit `\\`.
- `host` et `port` sont ceux de `DATABASE_URL`, vus **depuis un conteneur** : `localhost` ou `127.0.0.1` y désigne le conteneur, non le NAS — prenez l'IP LAN du NAS (dans `DATABASE_URL` aussi, pour la même raison). Si la base est un conteneur du réseau `frontend`, `host` est son nom.
- Le **paquet MariaDB 10 de DSM** écoute par défaut sur le port **3307** (et sur un socket local, que le conteneur ne voit pas) — fait de la documentation Synology, que la recette de ce manuel, menée sur conteneurs, ne rejoue pas. Sa connexion TCP/IP peut être désactivée (case à cocher dans l'application MariaDB 10), et le pare-feu de DSM (*Panneau de configuration* → *Sécurité* → *Pare-feu*) peut refuser le sous-réseau Docker : le conteneur n'atteint alors pas la base, et le dump échoue.
- Sans l'image `mariadb:10.11` et hors ligne, le dump échoue — voulu : un échec visible vaut

mieux qu'une sauvegarde sans base.

- Le schéma de Kesh n'a ni routine, ni déclencheur, ni événement, ni vue : le dump ne prend donc ni `--routines` ni `--events`.
- Les comptes vivent dans la base `mysql` du serveur, **pas** dans le dump. Sur un serveur MariaDB neuf (NAS perdu, base recréée), créez d'abord le compte Kesh (§5.3.2) et le compte de sauvegarde (ci-dessus), puis rechargez.

Le script de dump et sa tâche planifiée Enregistrez ce script sous `/volume1/docker/kesh/kesh-dump.sh`

```
#!/bin/bash
# /volume1/docker/kesh/kesh-dump.sh -- dump nocturne, avant Hyper Backup
set -euo pipefail
umask 077 # dump lisible de root seul (secrets)
export PATH=/usr/local/bin:/usr/bin:/bin
BASE=kesh # la base de DATABASE_URL
cd /volume1/docker/kesh/dump
trap 'rm -f kesh_pre_backup.sql.gz.tmp' EXIT # un dump raté ne laisse rien

docker run --rm --network frontend \
  -v /volume1/docker/kesh/kesh-dump.cnf:/etc/kesh-dump.cnf:ro \
  mariadb:10.11 \
  mariadb-dump --defaults-extra-file=/etc/kesh-dump.cnf \
  --single-transaction --add-drop-database --databases "$BASE" \
  | gzip > kesh_pre_backup.sql.gz.tmp
mv kesh_pre_backup.sql.gz.tmp kesh_pre_backup.sql.gz
sha256sum kesh_pre_backup.sql.gz > kesh_pre_backup.sql.gz.sha256
```

Puis créez la tâche : *Panneau de configuration* → *Planificateur de tâches* → *Créer* → *Tâche planifiée* → *Script défini par l'utilisateur*; onglet *Général* : utilisateur **root**; onglet *Programmer* : tous les jours à **1h30**, avant la tâche Hyper Backup (2h) — le dump prend quelques secondes à quelques minutes, laissez la marge; onglet *Paramètres de tâche* : commande `bash /volume1/docker/kesh/kesh-dump.sh`, et cochez l'envoi des détails d'exécution par e-mail *uniquement en cas d'arrêt anormal* : c'est le seul signal d'un dump raté. Lancez-la une première fois à la main (*Exécuter*) et lisez son résultat (*Action* → *Afficher le résultat*).

Vérifier que le dump du jour a été pris. Dans le Planificateur, le dernier résultat de la tâche doit être normal. Dans Hyper Backup, l'*Explorateur de sauvegardes* de la version de la nuit doit montrer `dump/kesh_pre_backup.sql.gz` daté du jour à 1h30 environ — un fichier daté de la veille signale un dump raté. Depuis une session root : `ls -l /volume1/docker/kesh/dump`. Lancer Hyper Backup à la fin du script par une commande de DSM n'est pas documenté par Synology : ce manuel ne le prescrit pas.

Ne supprimez pas le dump après la sauvegarde : il doit **rester** dans `dump/`, car c'est lui que capturent les snapshots horaires et Hyper Backup. Le supprimer retirerait la base de presque tous les snapshots : la recovery (§7.8.2, *Recovery depuis Snapshot*) n'aurait plus rien à recharger. Pour économiser l'espace, réduisez plutôt la rotation des snapshots.

i Note

Avec `docker-compose.prod.yml`, la base n'est **pas** sous `/volume1/docker/kesh` : sans la tâche de dump, Hyper Backup ne la sauvegarde que si son répertoire de données est lui-même dans la tâche. Le dump planifié est donc la voie normale, quelle que soit la charge. Hyper Backup ne sauvegarde pas la base au titre du dossier du compose ; si vous cochez aussi le paquet MariaDB 10 de DSM parmi les *Applications* de la tâche (possibilité de DSM, que la recette de ce manuel ne vérifie pas), cette copie s'ajoute au dump sans le remplacer : la voie que décrit ce manuel, et qu'il a éprouvée, est le dump.

7.8.2 Snapshot Replication — recovery instantané point-in-time

Snapshot Replication fournit des snapshots Btrfs natifs du volume `/volume1/` avec recovery quasi-instantané (< 1 minute) et point-in-time precision (jusqu'à 1 snapshot par 5 minutes). Le snapshot du dossier partagé `docker` couvre le dossier du compose — documents, logs, `.env`, `backup/`, le dump du jour —, pas la base. Pour les **fichiers** de ce dossier, il protège contre les erreurs humaines récentes (document supprimé, `.env` écrasé) qu'Hyper Backup serait trop coûteux à restaurer (téléchargement cloud + déchiffrement + restore). Pour la **base**, le point de restauration est celui du **dernier dump**, à la fréquence de la tâche de dump ; pour le resserrer, programmez la même tâche plus souvent (par exemple toutes les heures, aux heures des snapshots).

Prérequis Snapshot Replication

- Modèle NAS supportant **Btrfs** (DS220+, DS720+, DS920+, DS1522+, DS1821+ — vérifier la liste officielle Synology).
- Volume principal formaté en **Btrfs** (pas EXT4). Si déjà en EXT4, migration nécessaire (sauvegarde complète + recréation du volume + restore — opération longue, planifiée hors période d'activité).
- Paquet **Snapshot Replication** installé via Package Center.

Configuration Snapshot Replication pour Kesh

1. Ouvrir **Snapshot Replication** → onglet **Snapshots**.
2. Sélectionner le dossier partagé `docker` (qui contient `/volume1/docker/kesh`).
3. **Paramètres** → onglet **Snapshot** → activer *Programmer la création de snapshots automatiques*.
4. Fréquence recommandée : **toutes les heures, du lundi au vendredi 6h-22h** (couvre les périodes de saisie active fiduciaire).
5. Rotation : conserver **72 snapshots horaires** (3 jours d'historique) + **14 snapshots quotidiens** + **12 snapshots hebdomadaires**.
6. Laissez *Visible aux utilisateurs* **désactivé** : le dossier `docker` contient le dump, `backup/` et les fichiers d'identifiants, c'est-à-dire des secrets ; la restauration passe par l'administrateur.

Recovery depuis Snapshot Le snapshot rend le dossier du compose, dump de la base compris; il ne rend pas la base. Sans le rechargement du dump (étape 4), Kesh redémarre sur la base actuelle, et aucune écriture n'est restaurée.

1. Snapshot Replication → **Snapshots** → sélectionner le dossier `docker`.
2. Choisir le snapshot avec le timestamp pre-incident.
3. **Actions** → **Restaurer** → *Restaurer dans un nouveau dossier* (recommandé : l'état actuel, dump compris, reste intact pour comparaison) OU *Écraser le dossier original* (si certain). Ne copiez pas le dump restauré sur celui de `dump/` : le script ci-dessous le lit là où il est.
4. Recharger le dump par le script ci-dessous, en session root (`sudo -i`), en lui donnant le dossier `dump/` restauré : `bash /volume1/docker/kesh/kesh-restore.sh <dossier dump restauré>`.

Le script s'arrête à la première erreur, **avant** toute écriture si l'empreinte ou l'archive est fausse ; il prend un dump de l'état courant de la base (`avant-restauration/`, conservé) avant d'arrêter Kesh et de recharger, par le compte Kesh (`kesh-restore.cnf`) — non par `kesh-dump.cnf`, dont le compte ne peut que lire. Enregistrez-le sous `/volume1/docker/kesh/kesh-restore.sh` :

```
#!/bin/bash
# /volume1/docker/kesh/kesh-restore.sh <dossier du dump> -- recharge la base
set -euo pipefail
umask 077
export PATH=/usr/local/bin:/usr/bin:/bin
BASE=kesh # la base de DATABASE_URL
SOURCE=$1 # dossier qui contient
           kesh_pre_backup.sql.gz
cd "$SOURCE"
sha256sum -c kesh_pre_backup.sql.gz.sha256 # empreinte fausse : arrêt
ici
gzip -t kesh_pre_backup.sql.gz # archive tronquée : arrêt
ici
cd /volume1/docker/kesh
docker run --rm --network frontend \
  -v /volume1/docker/kesh/kesh-restore.cnf:/etc/kesh-restore.cnf:ro \
  mariadb:10.11 \
  mariadb-dump --defaults-extra-file=/etc/kesh-restore.cnf \
  --single-transaction --add-drop-database --databases "$BASE" \
  | gzip > "avant-restauration/kesh_$(date +%Y%m%d-%H%M%S).sql.gz"
docker compose stop kesh-api
gunzip -c "$SOURCE/kesh_pre_backup.sql.gz" | docker run --rm -i --network
frontend \
  -v /volume1/docker/kesh/kesh-restore.cnf:/etc/kesh-restore.cnf:ro \
  mariadb:10.11 \
  mariadb --defaults-extra-file=/etc/kesh-restore.cnf
docker compose up -d
```

Si le rechargement échoue après l'arrêt de Kesh, Kesh reste arrêté : rechargez le dump pris

dans `avant-restauration/` (même script, en copiant ce fichier et son empreinte — `sha256sum <fichier> > <fichier>.sha256` — dans un dossier à part, sous le nom `kesh_pre_backup.sql.gz`).

7.8.3 Stratégie 3-2-1 sur DSM

Combiner les outils pour atteindre la règle **3-2-1** (3 copies des données, sur 2 supports différents, dont 1 offsite) :

- **Copie 1 (primaire)** : volume Btrfs DSM avec Snapshot Replication actif — dossier du compose et dump de la base (recovery rapide erreurs récentes).
- **Copie 2 (locale, support différent)** : Hyper Backup vers HDD USB externe attaché au NAS, rotation Smart Recycle 30 jours + 12 mois.
- **Copie 3 (offsite)** : Hyper Backup vers cloud (Synology C2 Backup pour le confort, ou S3/Backblaze B2 pour le coût optimal), chiffré client-side.
- **Archivage long terme (OLiCo 10 ans)** : 1 dump annuel par year exporté manuellement sur média WORM (CD-R ou DVD-R inscriptible une fois) ou archivage tape LTO si volume.

Les copies 2 et 3 ont le périmètre de la copie 1. Après restauration du dossier du compose par Hyper Backup, la base se recharge par le même script : arrêt de `kesh-api`, rechargement du dump par `kesh-restore.cnf`, redémarrage (§7.8.2, *Recovery depuis Snapshot*). Sur un NAS ou un serveur MariaDB neuf, créez d'abord les deux comptes (ci-dessus).

! Attention

Test de restauration annuel obligatoire (cf. sous-section précédente OLiCo Art. 10 al. 1) : applicable aux backups DSM aussi. Inclure dans le procès-verbal annuel la mention explicite de la méthode utilisée (Hyper Backup, Snapshot Replication) et le temps de recovery mesuré (RTO).

7.9 Export/import d'installation via l'interface Kesh

Depuis la version v0.2, Kesh permet d'**exporter et de réimporter l'intégralité d'une installation** directement depuis l'interface web d'administration, sans accès SSH ni ligne de commande. L'export produit un fichier unique `.keshbackup` (conteneur compressé) contenant **toutes les données d'installation** : la société de l'installation, ses utilisateurs, et les données système — **à l'exception des fichiers** du répertoire des documents (justificatifs importés, PDF des factures émises), qui se sauvegardent à part (§7.3, issue #503). Cette fonction est **réservée au rôle Administrateur** et **inaccessible via une clé API (PAT)** — les opérations d'infrastructure ne transitent jamais par une intégration externe. La **sauvegarde de sécurité pré-import** fait exception à l'absence de SSH : la rapatrier pour la restaurer demande un accès SSH (voir plus bas).

i Note

À ne pas confondre avec l'**export global per-société** (menu **Administration** → **Export global**, fichier CSV/ZIP) qui n'extrait que les *données comptables* de la société pour transmission à un tiers. L'export/import `.keshbackup` décrit ici porte sur l'**installation complète** (société, utilisateurs, comptes et données système), pour la migration ou la restauration.

7.9.1 Exporter (sauvegarde complète)

Menu **Administration** → **Sauvegarde complète** (`/admin/backup`) → bouton « **Exporter toute l'installation** ». Le navigateur télécharge un fichier `kesh-installation-AAAA-MM-JJ.keshbackup`. Cette opération est en lecture seule (aucun effet de bord sur les données).

! Attention

Le fichier `.keshbackup` est un **secret** : il contient les **condensés (hash) des mots de passe** de tous les utilisateurs, les **condensés des clés API (PAT)** ainsi que les **jetons de session**. Il n'est **pas chiffré** par défaut. Conservez-le et transmettez-le comme une donnée hautement sensible — chiffrez-le pour tout transit hors de votre infrastructure contrôlée (par ex. `gpg` ou `age`).

7.9.2 Importer (restaurer la même installation)

Menu **Administration** → **Restaurer / Importer** (`/admin/restore`) → sélection du fichier `.keshbackup` → bouton « **Importer et remplacer l'installation** » → **confirmation forte** dans une fenêtre modale.

Cette fonction sert à restaurer votre installation — la remettre dans un état antérieur, ou la déplacer telle quelle vers un autre serveur. Ce n'est **pas** un outil pour verser les données d'une *autre* installation dans la vôtre : c'est techniquement possible, et les conséquences sur le journal d'audit sont décrites plus bas (section *Conformité OLICo*).

! Attention

Opération destructrice et irréversible. L'import **remplace l'intégralité des données** de l'installation actuelle par celles du fichier. À la fin, vous êtes **déconnecté** et devez vous reconnecter avec les **identifiants tels qu'ils étaient au moment de la sauvegarde** — tout compte créé, renommé ou dont le mot de passe a changé *depuis* cette sauvegarde revient à son état d'alors, et les comptes créés depuis n'existent plus. Avant toute suppression, Kesh crée **automatiquement une sauvegarde de l'état courant** côté serveur, en filet de sécurité — dans le dossier de sauvegarde `KESH_ADMIN_BACKUP_DIR`, défaut `/data/backup`, monté sur `./backup` par les compose fournis (voir *La sauvegarde de sécurité pré-import* ci-dessous).

L'import **refuse** le fichier et préserve l'installation si :

- le fichier est **corrompu** ou n'est pas un `.keshbackup` valide (contrôle d'intégrité SHA-256) ;
- le **schéma** de la sauvegarde est incompatible avec la version de Kesh installée ;
- la sauvegarde **exige une version minimale de Kesh supérieure** à celle de l'installation cible (protection anti-downgrade : ce seuil n'est relevé que par les migrations qui changent le schéma de façon incompatible) — mettez à jour Kesh avant de réimporter ;
- la **sauvegarde de sécurité** ne peut pas être prise ni écrite — dossier de sauvegarde non inscriptible ou disque plein, ou base de données momentanément inaccessible : le message le dit, **aucune sauvegarde n'est créée et rien n'est supprimé** ; les journaux du serveur (`docker compose logs kesh-api`) indiquent la cause.

La sauvegarde de sécurité pré-import Elle est écrite dans le dossier `backup/` du répertoire du compose (`/volume1/docker/kesh/backup` sur Synology), sous un nom qui commence par `kesh-pre-import-` (suivi de l'horodatage) et finit par `.keshbackup` : elle **survit** au conteneur, à sa recreation et à la mise à jour de l'image. Le fichier est créé en mode `0600`, propriétaire `root` (le conteneur tourne en `root`) ; sur Synology, les ACL du dossier partagé peuvent s'ajouter à ces droits — limite non mesurée. C'est un **secret** (mêmes contenus qu'un export). Il s'en ajoute une à chaque import, de la taille de l'installation : leur **purge** est à votre charge. Un fichier `...keshbackup.partial` est une écriture interrompue, **pas** une sauvegarde : il peut être supprimé.

Restaurer depuis elle. L'import se fait par le navigateur, et ni File Station, ni un partage SMB, ni un `scp` sans `sudo` ne lisent un fichier `root` en `0600`. Rapatriez-la donc en SSH vers un dossier partagé, depuis le répertoire du compose :

```
sudo cp backup/kesh-pre-import-<...>.keshbackup /volume1/<partage>/
sudo chown <utilisateur> /volume1/<partage>/kesh-pre-import-<...>.keshbackup
```

puis importez la copie par l'écran (ci-dessus), et **supprimez-la** ensuite : c'est un secret, et l'original reste dans `backup/`.

Exercices clôturés dans le désordre Une sauvegarde produite par une version antérieure à la v0.13.0 peut porter un exercice **ouvert suivi d'un exercice clôturé** — état que Kesh ne laisse plus se former, les exercices se clôturant désormais dans l'ordre. L'import **l'accepte** tel quel : il restaure votre installation, il ne la corrige pas. Après l'import, l'écran *Exercices comptables* affiche un **bandeau d'avertissement**, et **rien ne s'enregistre** dans l'exercice ouvert concerné — ni saisie, ni règlement, ni dévalidation, ni modification, ni suppression : le bilan étant cumulatif, toute écriture y changerait le bilan de l'exercice clôturé qui le suit. La réparation est celle que le bandeau indique, **dans cet ordre** :

1. clôturer l'exercice ouvert **le plus ancien** si ses comptes sont arrêtés (geste ouvert au Comptable), puis les exercices ouverts suivants, du plus ancien au plus récent ;

2. sinon, rouvrir les exercices clôturés postérieurs (administrateur, motif obligatoire), **en commençant par le plus récent** : rouvrir d'abord l'exercice clôturé le plus proche est refusé tant qu'un exercice plus récent est clôturé.

Kesh ne détecte pas cet état au démarrage et ne le répare pas d'office : la réparation est une décision comptable. Le manuel utilisateur la détaille (section *Exercices dans le désordre*).

Reprises de données rejouées à l'import Certaines mises à jour de Kesh ne font pas qu'ajouter des colonnes : elles **renseignent rétroactivement** des données existantes — par exemple les **rôles des comptes** du plan comptable (créances clients, TVA due, résultat de l'exercice...), ou le **compte de produit** des lignes de facture déjà validées.

Une sauvegarde produite **avant** l'une de ces mises à jour ne contient évidemment pas ces données. Or Kesh sait que la reprise a déjà tourné sur l'installation courante, et ne la relancerait donc pas : sans précaution, importer une telle sauvegarde ferait **disparaître ces données définitivement et sans aucun message**.

Kesh **rejoue donc ces reprises à la fin de chaque import**, dans la même opération que la restauration : soit tout réussit, soit rien n'est modifié. Trois propriétés à connaître :

- **Vos données ne sont jamais écrasées.** Si la sauvegarde contenait déjà l'information, elle fait foi : un rôle de compte ou un réglage que vous aviez ajusté à la main est conservé tel quel.
- **L'ordre est celui des mises à jour successives**, afin que le résultat soit identique à celui d'une installation qui aurait suivi le chemin normal des versions.
- **Le détail est tracé** dans le journal d'audit (opération `admin.full_import`) et dans le journal du serveur, qui indiquent quelles reprises ont été rejouées et combien de lignes elles ont touchées.

! Attention

Si vous avez restauré une sauvegarde **avant** la version qui introduit ce rejeu, réimporter **cette même sauvegarde** appliquera cette fois la reprise — mais **pesez-en le coût** : comme tout import, l'opération **remplace l'intégralité de l'installation** par le contenu du fichier. **Tout ce que vous avez saisi depuis ce restore serait perdu** (factures, écritures, contacts, paiements...). Ne relancez cet import que si **rien n'a été saisi entre-temps**. Dans le cas contraire, corrigez directement ce qui manque : les **rôles** depuis le plan comptable, le **compte de produit** d'une ligne depuis la facture concernée. Il n'existe pas de commande qui rejouerait les reprises **sans** restaurer : le rejeu est une étape de l'import, pas une opération autonome.

Variables d'environnement

- `KESH_ADMIN_IMPORT_MAX_MB` — taille maximale d'un fichier `.keshbackup` importé, en Mio (défaut **512**, bornes [1, 10240]).

- `KESH_ADMIN_EXPORT_INMEM_MB` — plafond d'assemblage en mémoire de l'export; au-delà, l'export est streamé via un fichier temporaire (défaut **50**, bornes [1, 2048]).
- `KESH_ADMIN_BACKUP_DIR` — dossier de sauvegarde de sécurité pré-import (défaut `/data/backup`, y compris quand la variable est vide, monté sur `./backup` par les compose fournis; créé en `0700` s'il n'existe pas). Une autre valeur doit être un chemin **monté**, sans quoi la sauvegarde retourne dans le conteneur; hors Docker, un dossier inscriptible.

7.9.3 Quelle méthode de sauvegarde choisir ?

Kesh propose trois approches complémentaires de sauvegarde. Choisissez selon le besoin :

Méthode	Périmètre	Usage recommandé
Hyper Backup DSM	Dossier du compose (documents, logs, <code>.env</code> , <code>backup/</code>) + dump de la base par une tâche planifiée (cf. 7.8)	Sauvegarde planifiée automatique quotidienne en production (NAS Synology), avec rotation et offsite.
<code>mariadb-dump</code> CLI	Dump SQL d'une base (niveau base de données)	Sauvegarde ponctuelle technique ou scriptée, pour un opérateur disposant d'un accès SSH.
Export/import <code>.keshbackup</code> (UI)	Installation Kesh complète, portable (niveau applicatif) — sans les fichiers du répertoire des documents (§7.3)	Migration entre instances ou restauration self-service sans SSH , déclenchée par un administrateur depuis l'interface.

! Astuce

Ces méthodes ne s'excluent pas : en production, conservez une sauvegarde système **automatique** (Hyper Backup avec sa tâche de dump, §7.8) comme filet quotidien, et utilisez l'export `.keshbackup` pour les **migrations ponctuelles** (changement de serveur, duplication d'environnement) où la portabilité applicative et l'absence d'accès SSH priment.

8 Mise à jour de Kesh

8.1 Process de release Kesh

Les versions de Kesh sont publiées sur GitHub avec un tag `vX.Y.Z` (versionnage sémantique). À chaque release :

- Une nouvelle image Docker `gcorbaz/kesh:X.Y.Z` et `gcorbaz/kesh:latest` sont poussées sur Docker Hub.
- Un changelog ([CHANGELOG.md](#)) liste les nouveautés, corrections de bugs et changements de compatibilité.
- Les manuels (admin, utilisateur, brochure marketing) sont rebuild en PDF et attachés au release GitHub.
- Les notes de breaking changes sont mises en évidence avec instructions de migration.

8.2 Suivre les notifications de release

- Activer les notifications GitHub : *Watch* le dépôt `guycorbaz/kesh` → *Custom* → *Releases*.
- Souscrire au flux RSS Atom des releases : <https://github.com/guycorbaz/kesh/releases.atom>.

8.3 Procédure de mise à jour standard

1. **Sauvegarder** la base de données et le volume `<projet>_kesh-mariadb-data` (cf. section précédente).
2. **Lire le changelog** de la nouvelle version pour identifier les breaking changes éventuels et les migrations nécessaires.
3. **Mettre à jour le fichier compose — obligatoire pour passer à la 0.13.0 (§8.3.1)** : re-télécharger le compose et y reporter ses adaptations locales, puis vérifier les placeholders de `.env` — dont les lignes `MARIADB_ROOT_PASSWORD` et `MARIADB_PASSWORD` : une valeur entre chevrons y est acceptée par Compose, et pour root rien d'autre ne la signale. Sur une base déjà créée, **ne remplacez pas** ces deux valeurs dans `.env` — ni des chevrons, ni `kesh_dev` : la base a gardé celles de sa création, et une valeur neuve n'y change rien (1045 pour Kesh, sauvegarde en échec pour root) ; changez-les par le §8.3.2, `ALTER USER` d'abord. Pour les versions suivantes : selon le changelog (nouvelles variables, changement d'image).
4. **Tirer la nouvelle image** : `docker compose pull kesh-api`.
5. **Redémarrer** : `docker compose up -d kesh-api`.
6. **Vérifier** les logs : `docker compose logs -f kesh-api` pour s'assurer que les migrations DB se sont bien déroulées.
7. **Tester** l'accès UI et les fonctionnalités critiques (login, saisie d'une écriture, génération d'une facture).

! Attention

Toujours sauvegarder **avant** la mise à jour. Une migration DB peut être irréversible. En cas de problème, la restauration depuis sauvegarde reste votre filet de sécurité.

8.3.1 Passer à la 0.13.0 : le compose et le fichier .env

Jusqu'à la 0.12, les compose fournis ne transmettaient pas à Kesh seize variables que `.env` proposait de régler (e-mails, mot de passe oublié, sortie de la démonstration, langue, plafonds...) : les modifier restait sans effet, sans aucun message. Le défaut est dans le **fichier compose que vous détenez** : la nouvelle image seule ne corrige rien, et le compose de la 0.13.0 s'emploie avec l'image 0.13.0.

! Attention

Relisez votre `.env` avant de mettre à jour : des lignes jusqu'ici sans effet le deviennent.

Refus de Compose (`docker-compose.yml`), **avant** Kesh : sans `MARIADB_ROOT_PASSWORD` ou `MARIADB_PASSWORD` dans `.env`, ou vide, la commande s'arrête **au terminal** en nommant la variable (`required variable ... is missing a value`) et aucun conteneur ne démarre — voir le paragraphe *MariaDB* ci-dessous.

Refus de démarrer — le message nomme la variable (`docker compose logs kesh-api`) :

- `KESH_SMTP_TLS` ou `KESH_FEATURE_FORGOT_PASSWORD` d'une valeur autre que `true/false/1/0` (espaces de tête et de fin ignorés), sur les deux compose — même sans le mot de passe oublié pour `KESH_SMTP_TLS` ;
- `KESH_COOKIE_SECURE` d'une valeur autre que `true/false/1/0` (espaces de tête et de fin ignorés), sur `docker-compose.prod.yml`, où il était ignoré ;
- `KESH_FEATURE_FORGOT_PASSWORD=true` avec l'une des cinq `KESH_SMTP_HOST/_USER/_PASSWORD/_FROM`, `KESH_PUBLIC_BASE_URL` absente, un `KESH_SMTP_FROM` mal formé (« Nom <email> ») ou un `KESH_SMTP_HOST` au format hôte:port ;
- `KESH_FEATURE_FORGOT_PASSWORD=true` avec une configuration SMTP complète dont le mailer ne se construit pas (adresse `KESH_SMTP_FROM` refusée, échec de l'initialisation STARTTLS) : « le build du mailer SMTP a échoué ... abandon » (sans le mot de passe oublié, le même échec rend seulement l'envoi de factures indisponible) ;
- `KESH_JWT_SECRET` resté au placeholder du gabarit ou à une valeur entre chevrons ; `KESH_ADMIN_PASSWORD` décommenté sans remplacer le sien, ou recopié d'un exemple de ce manuel (`<mot de passe fort, ...>`, `<nouveau-mdp-12+>`) ;
- `MARIADB_PASSWORD` changé dans `.env` sans `ALTER USER` : « Base de données indisponible ... 1045 ... », suivi d'un indice (§8.3.2).

Effets sans refus : `KESH_PRODUCTION_RESET` posé lors d'une ancienne démonstration réautorise, **sur une installation restée en démonstration**, la réinitialisation, qui vide toutes les données de la société ; les quatre `KESH_SMTP_HOST/_USER/_PASSWORD/_FROM` posées « pour plus tard » activent l'envoi de factures par e-mail, et avec `KESH_FEATURE_FORGOT_PASSWORD=true` (et `KESH_PUBLIC_BASE_URL`) la réinitialisation du mot de passe par e-mail — quiconque connaît l'adresse d'un utilisateur peut alors demander un lien ; `KESH_COOKIE_SECURE=false` sur `docker-compose.prod.yml` émet des cookies sans Secure ; `KESH_ADMIN_BACKUP_DIR` (un chemin non monté fait écrire dans le conteneur ; sans cette ligne, la sauvegarde pré-import va désormais, avec le compose de la 0.13.0, dans `./backup` — avec un compose gardé sans ce montage, dans `/data/backup` du conteneur, éphémère comme avant) ; `KESH_LANG` ; `KESH_PASSWORD_MIN_LENGTH` sur `docker-compose.yml` s'applique côté serveur, alors que trois écrans gardent 12 en dur : au-dessous de 12, les écrans refusent quand même ; au-dessus, l'écran accepte et le serveur refuse ; une ligne `KESH_LOG_FILE_PATH=` **vide** désactive désormais le journal fichier ; et plus généralement toute ligne décommentée de ces seize variables.

Un `.env` copié du gabarit sans modification ne change pas de comportement — les seules lignes actives du gabarit parmi ces variables valent leur défaut (`KESH_LANG=fr`, `KESH_PASSWORD_MIN_LENGTH=12`, `KESH_BANK_IMPORT_MAX_MB=10`, `KESH_COOKIE_SECURE=true`) — **sauf** le secret JWT laissé au placeholder, désormais refusé, **et** les mots de passe MariaDB avec `docker-compose.yml` : copié du gabarit **0.13.0**, où ils sont commentés et sans valeur, il fait refuser Compose ; copié d'un gabarit **antérieur**, qui posait `kesh_dev`, il fait émettre à Kesh l'avertissement « mot de passe publié ». Un `KESH_ADMIN_PASSWORD` absent ou vide ne change rien avec `docker-compose.prod.yml`, et cesse de faire refuser le démarrage avec `docker-compose.yml`.

Voie recommandée : re-télécharger le compose — `docker-compose.yml`, ou, sur Synology, `docker-compose.prod.yml` enregistré sous `docker-compose.yml` (§4.5) — **et y reporter vos seules adaptations locales** (port 8080:80, réseau, chemins des montages). Un montage modifié à la main et non reporté revient à `./documents` (ou `./inbox`, `./log`) : justificatifs et PDF paraissent alors disparus — ils sont toujours dans l'ancien dossier. Les commandes s'exécutent depuis le répertoire du compose (`/volume1/docker/kesh` sur Synology) ; avec un autre nom de fichier, ajoutez `-f <son fichier>` à chaque commande `docker compose`. Puis `docker compose pull` **avant** `docker compose up -d`. Le compose de la 0.13.0 monte en outre `./backup` (sauvegarde de sécurité pré-import, §7.9) : le re-télécharger suffit pour ce montage.

MariaDB (`docker-compose.yml`) : port fermé, mots de passe obligatoires.

Le compose 0.13.0 ne publie plus le port 3306 de MariaDB — le re-télécharger

suffit à le fermer; un outil de l'hôte qui s'y connectait passe désormais par `docker compose exec mariadb ...` — et n'a plus de mots de passe MariaDB par défaut. Avec un `.env` sans `MARIADB_ROOT_PASSWORD` ou sans `MARIADB_PASSWORD`, **Compose refuse** `config`, `pull` et `up` en nommant la variable, au terminal; `ps`, `logs`, `exec` et `stop` restent utilisables (mesuré avec Docker Compose 2.40), de quoi diagnostiquer et arrêter. Écrivez alors dans `.env` la valeur avec laquelle la base a été créée, jamais une neuve : si vous n'en aviez jamais posé, ce sont les anciennes valeurs par défaut, `MARIADB_ROOT_PASSWORD=kesh_dev_root` et `MARIADB_PASSWORD=kesh_dev`. Une valeur neuve ne change pas le mot de passe enregistré : pour `MARIADB_PASSWORD`, Kesh échoue à se connecter (erreur 1045, avec un indice); pour `MARIADB_ROOT_PASSWORD`, rien ne le signale, mais la sauvegarde nocturne échoue. Démarrez, puis **changez ces mots de passe** par la procédure du §8.3.2 : publiés dans le dépôt Kesh, ils ne protègent rien — Kesh avertit au démarrage tant que celui de `DATABASE_URL` vaut `kesh_dev` ou une valeur gabarit `<...>`. Terminez par le **contrôle de fin de mise à jour** ci-dessous.

Vérifier les placeholders (Kesh 0.13.0 refuse de démarrer si `KESH_JWT_SECRET` ou `KESH_ADMIN_PASSWORD` est resté au placeholder `<GENERATE_ME: ...>` du gabarit ou à une valeur entre chevrons `<...>`). Le contrôle se demande à Compose lui-même, le compose final en place — Compose lit `.env` à sa façon (`export`, espaces autour de `=`, guillemets, fins de ligne Windows), qu'aucun `grep` sur `.env` n'imité sûrement. Les deux mots de passe MariaDB posés (paragraphe précédent), demandez d'abord à Compose s'il lit le fichier, puis cherchez les placeholders :

```
sudo docker compose config -q && echo 'compose lisible'
sudo docker compose config \
  | grep -iE 'KESH_(JWT_SECRET|ADMIN_PASSWORD): .? *(<|.generate_me)'
```

La première commande **doit** afficher `compose lisible` : si Compose refuse le fichier, la seconde ne reçoit rien et reste muette — muette comme un succès. La seconde doit rester **muette**. Dans le cas qu'elle détecte, elle affiche la ligne trouvée — un placeholder, non un secret. Elle est un sur-ensemble du refus de Kesh (une valeur qui commence par `<` sans finir par `>` y apparaît aussi, que Kesh accepte). Si elle montre `KESH_JWT_SECRET` : générez le secret (`openssl rand -hex 32`) **avant** le `up -d` — les sessions ouvertes sont alors perdues, chacun se reconnecte. Si elle montre `KESH_ADMIN_PASSWORD` : choisissez un vrai mot de passe (≥ 12 caractères) ou recommencez la ligne (onboarding `/setup` sur une base vide, sans effet sur une base qui a déjà un administrateur). Faute de quoi Kesh refuse de démarrer. Ce contrôle ne voit pas les mots de passe MariaDB : relisez à l'éditeur les lignes `MARIADB_ROOT_PASSWORD` et `MARIADB_PASSWORD` de `.env` — pour un root resté au gabarit `<...>`, c'est le seul signal. **Contrairement aux deux variables précédentes, ne remplacez pas ces chevrons dans `.env`** : la base a été créée avec cette chaîne et la garde; changez le mot de passe par le §8.3.2 (`ALTER USER` d'abord, `.env` ensuite).

Pour qui garde son fichier compose.

Cinq gestes pour `docker-compose.yml`, trois pour `docker-compose.prod.yml`. Les deux pre-

miers, dans les deux fichiers, sous `environment` : du service `kesh-api` ; le dernier, dans les deux fichiers aussi, sous ses `volumes` :

1. **ajouter** les lignes **nouvelles**, et elles seules — pour `docker-compose.yml` :

```
KESH_LANG: ${KESH_LANG:-}
KESH_PASSWORD_MIN_LENGTH: ${KESH_PASSWORD_MIN_LENGTH:-}
KESH_BANK_IMPORT_MAX_MB: ${KESH_BANK_IMPORT_MAX_MB:-}
KESH_SMTP_HOST: ${KESH_SMTP_HOST:-}
KESH_SMTP_PORT: ${KESH_SMTP_PORT:-}
KESH_SMTP_USER: ${KESH_SMTP_USER:-}
KESH_SMTP_PASSWORD: ${KESH_SMTP_PASSWORD:-}
KESH_SMTP_FROM: ${KESH_SMTP_FROM:-}
KESH_SMTP_TLS: ${KESH_SMTP_TLS:-}
KESH_PUBLIC_BASE_URL: ${KESH_PUBLIC_BASE_URL:-}
KESH_FEATURE_FORGOT_PASSWORD: ${KESH_FEATURE_FORGOT_PASSWORD:-}
KESH_PRODUCTION_RESET: ${KESH_PRODUCTION_RESET:-}
KESH_ADMIN_BACKUP_DIR: ${KESH_ADMIN_BACKUP_DIR:-}
KESH_ADMIN_EXPORT_INMEM_MB: ${KESH_ADMIN_EXPORT_INMEM_MB:-}
KESH_ADMIN_IMPORT_MAX_MB: ${KESH_ADMIN_IMPORT_MAX_MB:-}
```

pour `docker-compose.prod.yml` :

```
KESH_COOKIE_SECURE: ${KESH_COOKIE_SECURE:-}
KESH_SMTP_HOST: ${KESH_SMTP_HOST:-}
KESH_SMTP_PORT: ${KESH_SMTP_PORT:-}
KESH_SMTP_USER: ${KESH_SMTP_USER:-}
KESH_SMTP_PASSWORD: ${KESH_SMTP_PASSWORD:-}
KESH_SMTP_FROM: ${KESH_SMTP_FROM:-}
KESH_SMTP_TLS: ${KESH_SMTP_TLS:-}
KESH_PUBLIC_BASE_URL: ${KESH_PUBLIC_BASE_URL:-}
KESH_FEATURE_FORGOT_PASSWORD: ${KESH_FEATURE_FORGOT_PASSWORD:-}
KESH_PRODUCTION_RESET: ${KESH_PRODUCTION_RESET:-}
KESH_ADMIN_BACKUP_DIR: ${KESH_ADMIN_BACKUP_DIR:-}
KESH_ADMIN_EXPORT_INMEM_MB: ${KESH_ADMIN_EXPORT_INMEM_MB:-}
KESH_ADMIN_IMPORT_MAX_MB: ${KESH_ADMIN_IMPORT_MAX_MB:-}
```

2. **remplacer** les lignes **existantes** qui changent de forme, dans les deux compose — ligne remplacée, jamais dupliquée :

```
KESH_ADMIN_PASSWORD: ${KESH_ADMIN_PASSWORD:-}
KESH_LOG_FILE_PATH: ${KESH_LOG_FILE_PATH-/var/log/kesh/kesh.log}
```

`docker-compose.yml` seulement, service `mariadb` : **retirer** la clé `ports` : et sa ligne — `"3306:3306"` ;

- 3.

4. `docker-compose.yml` seulement : **remplacer** les fragments `:-kesh_dev_root` et `:-kesh_dev` — **trois occurrences** : `MARIADB_ROOT_PASSWORD` et `MARIADB_PASSWORD` du

`service mariadb, DATABASE_URL` de `kesh-api` — par `: ?` suivi d'un message, par exemple :

```
MARIADB_PASSWORD: ${MARIADB_PASSWORD:?à poser dans .env}
```

Exemple abrégé : le message du compose re-téléchargé ajoute « installation existante, reprendre la valeur avec laquelle la base a été créée, jamais une neuve », que Compose affiche au refus. Le plus sûr est de recopier ces trois lignes, message compris, depuis le compose re-téléchargé.

5. **ajouter** sous `volumes` : du service `kesh-api`, dans les deux compose, le montage de la sauvegarde de sécurité pré-import — sans lui, elle reste dans le conteneur et disparaît à sa recréation :

```
- ./backup:/data/backup
```

Puis `docker compose config -q` (qui refuse une clé en double, et l'absence des deux mots de passe MariaDB), vérifier que `docker compose config` montre `KESH_SMTP_HOST` et `KESH_PRODUCTION_RESET`, et **aucune** entrée `ports` sous le service `mariadb` (Compose l'écrit sous sa forme développée, `published: "3306"`), que la commande ci-dessous affiche 1 — elle compte la **cible** du montage, pas la variable, qu'un `.env` peut poser sans montage —, et `docker compose up -d`.

```
sudo docker compose config | grep -c 'target: /data/backup'
```

Contrôle de fin de mise à jour (MariaDB, `docker-compose.yml`).

Il se fait **sur la base**, pas sur `.env` : un `grep` sur `docker compose config` ne lit que `.env`, et resterait muet alors que la base garde l'ancien mot de passe. Une fois les mots de passe changés (§8.3.2) et `docker compose up -d` passé :

```
# Anciens mots de passe publiés : chaque commande doit être REFUSÉE (1045)
sudo docker compose exec -T mariadb mariadb -uroot -pkesh_dev_root \
  -e 'SELECT CURRENT_USER()'
sudo docker compose exec -T mariadb mariadb --protocol=TCP \
  -h 127.0.0.1 -uroot -pkesh_dev_root -e 'SELECT CURRENT_USER()'
sudo docker compose exec -T mariadb mariadb --protocol=TCP \
  -h 127.0.0.1 -ukesh -pkesh_dev -e 'SELECT CURRENT_USER()'
# Mots de passe de .env, lus dans le conteneur : chaque commande RÉUSSIT
sudo docker compose exec -T mariadb sh -c 'mariadb -uroot \
  -p"$MARIADB_ROOT_PASSWORD" -e "SELECT CURRENT_USER()"'
sudo docker compose exec -T mariadb sh -c 'mariadb --protocol=TCP \
  -h 127.0.0.1 -uroot -p"$MARIADB_ROOT_PASSWORD" \
  -e "SELECT CURRENT_USER()"'
sudo docker compose exec -T mariadb sh -c 'mariadb --protocol=TCP \
  -h 127.0.0.1 -ukesh -p"$MARIADB_PASSWORD" -e "SELECT CURRENT_USER()"'
```

Les trois premières doivent échouer (ERROR 1045), les trois dernières afficher le compte qui

a répondu. Sans `--protocol=TCP`, la connexion passe par le socket et atteint le compte `root@localhost`; en TCP vers `127.0.0.1`, le compte `root@%` (le serveur ne résout pas les noms) — MariaDB exige le mot de passe dans les deux cas (mesuré, MariaDB 10.11). D'où les deux contrôles de root. Puis **lancez une fois à la main le script de sauvegarde** (§7.4) : c'est le seul moyen de voir qu'un mot de passe root changé dans `.env` sans `ALTER USER` le fait échouer.

8.3.2 Changer un mot de passe MariaDB

Avec `docker-compose.yml`. MariaDB ne relit pas `MARIADB_ROOT_PASSWORD` ni `MARIADB_PASSWORD` après la création de la base : on change le mot de passe **dans la base d'abord** (`ALTER USER`), **puis** dans `.env`, **puis** on recrée les conteneurs. L'ordre inverse produit un refus 1045 de Kesh, ou une sauvegarde en échec pour root. Lister d'abord les comptes :

```
cd /opt/kesh                                # répertoire du compose
sudo docker compose exec -T mariadb sh -c \
    'mariadb -uroot -p"$MARIADB_ROOT_PASSWORD" \
    -e "SELECT user, host FROM mysql.user"'
```

L'image MariaDB crée deux comptes root (`root@localhost` et `root@%`) et le compte applicatif, `kesh@%` par défaut — son nom est celui de `MARIADB_USER`, que la recette lit dans le conteneur ; il faut une ligne `ALTER USER` par compte root listé. Puis :

```
cd /opt/kesh
( set -e                                # s'arrête à la première erreur
  COMPTE=$(sudo docker compose exec -T mariadb printenv MARIADB_USER)
  NEW_ROOT=$(openssl rand -hex 32)       # jamais tapés : absents de
  NEW_KESH=$(openssl rand -hex 32)       # l'historique du shell
  # Affichés AVANT d'être appliqués : recopiez-les dans .env, à
  # l'éditeur, sans enregistrer encore
  echo "MARIADB_ROOT_PASSWORD=$NEW_ROOT"
  echo "MARIADB_PASSWORD=$NEW_KESH"
  read -r -p 'Recopiés ? Entrée pour appliquer, Ctrl-C pour renoncer '
  sudo docker compose stop kesh-api
  sudo docker compose exec -T mariadb \
    sh -c 'mariadb -uroot -p"$MARIADB_ROOT_PASSWORD" ' <<SQL
ALTER USER '$COMPTE'@'%' IDENTIFIED BY '$NEW_KESH';
ALTER USER IF EXISTS 'root'@'localhost' IDENTIFIED BY '$NEW_ROOT';
ALTER USER IF EXISTS 'root'@'%' IDENTIFIED BY '$NEW_ROOT';
SQL
  echo 'CHANGÉS DANS LA BASE : enregistrez .env'
)
```

Puis, **seulement** après CHANGÉS DANS LA BASE et `.env` enregistré (commande à part : collée avec le bloc précédent, elle partirait avant) :

```
sudo docker compose up -d
```

Kesh est arrêté d'abord : entre l'`ALTER USER` et le `up -d`, ses connexions neuves échoueraient. Les nouveaux mots de passe ne figurent ni dans l'historique du shell (engendrés dans une variable) ni dans la liste des processus de l'hôte (transmis à `mariadb` par l'entrée standard, jamais en argument); les `echo` les affichent **avant** l'`ALTER USER`, pour que vous les recopiez dans `.env` — pas par `sed -i`, qui les mettrait en argument : une session coupée après le changement ne les emporte pas. Les parenthèses et `set -e` arrêtent la recette à la première erreur sans fermer votre terminal ; `mariadb` s'arrête de même à la première instruction refusée, et le compte applicatif — seule ligne sans `IF EXISTS` — passe en premier : s'il est refusé, rien n'a changé. Sans la ligne `CHANGÉS DANS LA BASE`, n'enregistrez pas `.env` ; lisez l'erreur, puis `sudo docker compose start kesh-api`. La connexion utilise encore l'**ancien** mot de passe root, lu dans l'environnement du conteneur : celui-ci ne prend les nouvelles valeurs qu'au `up -d`, qui recrée le conteneur `mariadb` (le volume, donc la base, est conservé ; un `up -d kesh-api` le recrée aussi). Pour ne changer que l'un des deux, retirez pour l'autre sa génération (`NEW_...`), son `echo` et ses lignes `ALTER USER`, et laissez sa ligne de `.env` intacte. Terminez par le contrôle de fin de mise à jour (§8.3.1) et un lancement à la main du script de sauvegarde.

Vérification fonctionnelle : `curl http://127.0.0.1/health` montre `"smtpConfigured":true` si le SMTP est posé.

8.4 Mise à jour majeure (changement de version X)

Une mise à jour majeure (par exemple `v0.1` → `v1.0`) peut comporter des breaking changes. Procédure recommandée :

1. Lire intégralement le changelog et les notes de migration.
2. Provisionner une instance de test en parallèle de la production.
3. Restaurer une copie récente de la base de production sur l'instance de test.
4. Faire la mise à jour sur l'instance de test.
5. Valider toutes les fonctionnalités critiques sur l'instance de test.
6. Sauvegarder la production, puis appliquer la mise à jour.
7. Surveiller les logs pendant 24–48h post-update.

8.5 Rollback en cas d'échec

Si une mise à jour échoue ou révèle un bug bloquant :

```
# 1. Arrêter le service
docker compose stop kesh-api

# 2. Restaurer la version précédente de l'image
```

```
docker compose down
# Editer docker-compose.yml : image: gcorbaz/kesh:0.1.0 (ancien tag)
docker compose up -d

# 3. Si les migrations DB ont ete appliquees et sont irreversibles, restaurer
# la sauvegarde DB d'avant la mise a jour (cf. section Restauration).
```

9 Sécurité

9.1 HTTPS obligatoire en production

Kesh **doit** être accessible uniquement via HTTPS en production. Le mode HTTP plain doit être réservé strictement au développement local.

Configurations recommandées du reverse proxy :

- TLS 1.2 minimum, TLS 1.3 préféré.
- Suites de chiffrement modernes uniquement (cf. [Mozilla SSL Config Generator](#)).
- HSTS activé avec `max-age=31536000; includeSubDomains`.
- Certificats Let's Encrypt (renouvellement automatique via Certbot ou Caddy intégré).
- Headers de sécurité : `X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin`.

9.2 Gestion des secrets

- **Jamais committer** `.env` dans Git. Ajouter à `.gitignore`.
- Utiliser un secrets manager (HashiCorp Vault, AWS Secrets Manager, Doppler, ou simplement `systemd-creds` sous Linux) pour les déploiements multi-instances.
- Permissions strictes sur `.env` : `chmod 600 .env` (lecture/écriture propriétaire uniquement).
- Rotation périodique du `KESH_JWT_SECRET` : tous les 6–12 mois en production. Une rotation invalide tous les tokens existants, forçant les utilisateurs à se reconnecter.

9.3 Authentification JWT + refresh tokens

Kesh utilise un modèle JWT avec rotation de refresh tokens :

- L'access token (JWT court, 15 min par défaut) est envoyé en cookie `HttpOnly; Secure; SameSite=Lax`.
- Le refresh token (durée 30 jours par défaut) est stocké en DB (table `refresh_tokens`) avec

hash SHA-256 (jamais stocké en clair).

- À chaque refresh, le token précédent est invalidé (rotation), une trace est conservée pour détection d'attaque par replay.
- La déconnexion révoque tous les refresh tokens de la session.

9.4 Clés API (PAT) — accès programmatique

En complément de l'authentification web par cookie, Kesh permet à des **intégrations externes** (IA, scripts, ETL, dashboards BI, ERP) de consommer l'API via des **clés d'accès personnelles** (PAT — *Personal Access Token*). Une clé s'utilise sur les routes `/api/v1/*` existantes, présentée dans l'en-tête `Authorization: Bearer kesh_pat_...`

- **Gestion** : les clés se créent et se révoquent depuis l'interface web, page **Paramètres → Clés API** (`/settings/api-keys`), accessible aux rôles Comptable et Administrateur. La gestion des clés est **impossible via l'API elle-même** (protection anti-auto-propagation : une clé compromise ne peut pas en créer d'autres).
- **Administration** : plus largement, **aucune route réservée au rôle Administrateur n'est atteignable avec une clé API**, quel que soit le rôle de son créateur — gestion des utilisateurs et des mots de passe, paramètres de facturation et de relance, taux de TVA, niveaux de rappel, modèles d'e-mail, coordonnées de l'entreprise, suppression de facture, export et import d'installation, réouverture d'exercice. La réponse est `403 API_KEY_ADMIN_FORBIDDEN`. Ces opérations se font dans l'interface web. Cette fermeture est livrée depuis la v0.10.0 — voir l'encadré « *Dans quelle version ?* » plus bas.
- **Journal d'audit** : la **consultation du journal** — la liste, le vocabulaire et l'export CSV (`/api/v1/audit-log*`) — est elle aussi **fermée aux clés API**, y compris en lecture seule et y compris pour une clé créée par un Administrateur. *La réponse porte ici un **autre** code : `403 API_KEY_MANAGEMENT_FORBIDDEN`, et non `API_KEY_ADMIN_FORBIDDEN` — le journal n'est pas une route d'administration. Le libellé parle de « gestion de clés » pour des raisons historiques ; sur ces trois routes, il signifie simplement qu'une clé n'y a pas accès. Ne cherchez pas le défaut dans la configuration de la clé.* Le journal d'audit se lit dans l'interface web, par un Comptable ou un Administrateur.
- **Ce qu'une clé `read-write` peut faire depuis la v0.12.1, et qu'elle ne pouvait pas avant : dévalider une facture**, donc **supprimer l'écriture comptable** que sa validation avait créée. La route n'est pas réservée à l'administrateur — elle est ouverte au Comptable, et donc aux clés — par un choix délibéré, aligné sur les API de facturation comparables. Les neuf refus de la dévalidation s'y appliquent intégralement (exercice clos, exercice suivi d'un exercice clôturé, verrou de période, facture réglée, créditée, relancée, envoyée, rapprochée, contre-passée), et chaque dévalidation est journalisée sous son propre nom. Si cette capacité vous paraît trop large pour une intégration, donnez-lui une clé en *lecture seule*.
- **Depuis la v0.13.0, une clé `read-write` peut aussi modifier une écriture** saisie à la main (`PUT /api/v1/journal-entries/{id}`), sous les mêmes gardes qu'à l'écran : exercice ouvert, aucun exercice postérieur clôturé, aucune pièce propriétaire, période non ver-

rouillée, mêmes contrôles qu'à la saisie, aucune ligne lettrée (409 `ENTRY_LETTERED` : elle se délètte d'abord). Chaque modification est tracée avec l'état avant et après, **au nom de la clé**. Dans le **même cadre**, elle peut **supprimer** une écriture saisie à la main (`DELETE /api/v1/journal-entries/{id}`) : la trace `journal_entry.deleted` porte le contenu complet de l'écriture, **au nom de la clé**, et le numéro n'est jamais réattribué.

- **Portée** : chaque clé est en *lecture seule* (`read` — méthodes `GET/HEAD/OPTIONS` ; une lecture n'écrit rien, à une exception près : le **premier téléchargement** du PDF d'une facture — y compris une simple requête `HEAD` — le **fige** et l'inscrit au journal d'audit au nom de la clé) ou *lecture-écriture* (`read-write` — toutes les méthodes). La permission effective est l'intersection de la portée de la clé et du rôle de son créateur — **et l'administration en est exclue dans tous les cas** (point *Administration* ci-dessus), **de même que le journal d'audit** (point précédent).
- **Périmètre** : une clé est liée à **une seule entreprise** (*company*) et agit au nom de son créateur (rôle relu en base à chaque requête — désactiver le créateur invalide ses clés immédiatement).
- **Stockage** : seul un condensé SHA-256 de la clé est conservé en base (table `api_keys`) ; le secret en clair n'est affiché qu'une seule fois, à la création.
- **Expiration / révocation** : une clé peut avoir une date d'expiration optionnelle ; la révocation prend effet immédiatement.
- **Journal d'audit** : les mutations effectuées via une clé sont tracées dans `audit_log`. *L'attribution n'est pas encore complète : une quinzaine de dépôts de données inscrivent `actor_type = 'user'` au lieu de `'api_key'`, si bien qu'une action faite par une intégration peut être journalisée comme venant d'une personne. L'imputabilité humaine est préservée — c'est le créateur de la clé qui figure — mais l'information « c'était une intégration » se perd. Correction suivie à l'issue #431.*

! Attention

Moindre privilège. Une clé créée par un **Administrateur n'hérite pas** de ses pouvoirs d'administration : celle-ci est fermée aux clés API sans exception, de sorte que **révoquer une clé compromise suffit à arrêter l'incident**. Appliquez néanmoins le moindre privilège à ce que la clé peut faire : privilégiez la portée `read` lorsque la lecture suffit, et un créateur Comptable quand l'intégration n'a pas besoin d'écrire davantage. La v0.2 n'applique **pas** de limitation de débit (*rate-limiting*) par clé : utilisez l'expiration et la révocation pour limiter l'exposition.

! Attention

Dans quelle version ? Depuis la v0.10.0. Jusqu'à la **v0.9.0 incluse**, une clé `read-write` créée par un Administrateur atteint bien les routes d'administration — elle peut donc créer un compte administrateur, ce qui rend la révocation de la clé **inopérante**. Le correctif décrit au point **Administration** ci-dessus est livré par la **v0.10.0** (entrée #167 du [CHANGELOG.md](#)). **Si vous exploitez encore une version antérieure, traitez une clé d'origine Administrateur comme un secret d'administrateur** : ne l'utilisez que si l'intégration en a réellement besoin, préférez un créateur Comptable, et mettez Kesh à jour. Suivi GitHub [KF-036](#) / #167.

Le guide d'intégration complet (exemples `curl`, Python, JavaScript, MCP, table des codes d'erreur) est fourni dans le dépôt à `docs/api-external.md`.

9.5 RBAC et permissions

Le contrôle d'accès basé sur les rôles est appliqué **au niveau de chaque endpoint API** via un middleware Tower (Axum). Les trois rôles standards (cf. section Premier démarrage) sont définis dans le code (pas de rôles custom dynamiques v0.1).

9.6 Journal d'audit (`audit_log`)

La quasi-totalité des actions métier mutantes est enregistrée dans la table `audit_log` — 105 des 112 routes à verbe mutant au moment de la rédaction, dont, depuis la v0.13.0, la configuration d'une installation de production (une entrée par étape franchie) et le chargement des données de démonstration (une entrée de synthèse). *Les exceptions sont connues, comptées et suivies : cinq routes exemptées — la remise à zéro des données de démonstration (issue #434), et quatre gestes de session, la connexion, la déconnexion, le renouvellement de session et le changement de son propre mot de passe (issue #435) —, et deux autres routes qui n'écrivent rien parce qu'elles ne modifient rien : $105 + 5 + 2 = 112$. Limite connue, suivie par l'issue #538 : l'entrée du chargement des données de démonstration n'est écrite que lorsqu'il aboutit. Un chargement interrompu — ou refusé parce que la configuration de production a franchi l'étape entre-temps — laisse la société renommée, le plan comptable et l'exercice **sans entrée au journal**.* La liste complète, route par route, est tenue dans le dépôt et contrôlée par un test :

- Contrainte **insert-only** : aucune route API ne permet de modifier ou supprimer une entrée `audit_log`, et **aucune ne permet d'effacer la table** sur une installation finalisée. *La formulation antérieure ne parlait que d'« une entrée » : elle était exacte au mot près et trompeuse en substance, une route effaçant alors la table entière.*
- Champs : `id`, `user_id`, `actor_label` (le nom de l'auteur *au moment* de l'écriture), `actor_type`

et `actor_api_key_id`, `company_id` (la société de l'auteur au moment de l'écriture, vide quand elle est indéterminable), `action`, `entity_type`, `entity_id`, `details_json`, `created_at`.

Chaque entrée porte la société de son auteur. Le journal se **consulte** désormais depuis l'API : une liste paginée et filtrable, un vocabulaire des codes traduits, et un export CSV — réservés aux rôles **Comptable** et **Administrateur**, et refusés aux clés API. L'écran existe aussi : Administration → Journal d'audit, pour les mêmes rôles. La consultation ne filtre que votre société, et ne s'inscrit pas elle-même au journal.

- Conservation 10 ans (selon CO Art. 958f al. 1).

9.7 Conformité OLICo Art. 9 — intégrité des supports modifiables

Kesh met en place les éléments que l'OLICo Art. 9 al. 1.b attend d'un support modifiable :

- **Un verrou de période** arrête les écritures à une date choisie, indépendamment de la clôture annuelle : aucune écriture ne peut plus être datée d'une période déjà déclarée (typiquement un trimestre TVA déposé), par aucun chemin — ni la saisie manuelle, ni les flux automatiques — et aucune écriture de la période ne peut plus en être **supprimée** ni **modifiée**, pas même en **dévalidant** la facture qui la porte. Le **déverrouillage** est réservé à un administrateur, exige un **motif** et s'inscrit au journal d'audit ; une restauration de sauvegarde qui ferait reculer la borne y laisse également une trace (`books.restored`).
- **Une écriture se modifie tant que son exercice est ouvert** (depuis la v0.13.0) — si aucun exercice postérieur n'est clôturé, qu'aucune pièce ne la possède, hors période verrouillée et si aucune de ses lignes n'est lettrée (elle se délettre alors d'abord) — et **chaque modification est tracée** au journal d'audit avec l'état **avant et après**, lignes comprises, et son auteur (CO Art. 958f : l'exigence n'est pas qu'on ne se trompe jamais, c'est que la correction soit apparente). La conformité repose donc sur cette **trace**, sur la **clôture** et sur le **verrou de période**, qui figent l'écriture : ensuite, la correction passe par la **contre-passation**, qui laisse l'origine et sa correction dans les livres. La **suppression** d'une écriture suit le même cadre que sa modification (depuis la v0.13.0) : refusée dès que l'exercice ou un exercice postérieur est clôturé, qu'une pièce la possède, que sa date est verrouillée ou qu'une de ses lignes est lettrée, et **tracée** avec le contenu complet de l'écriture. Son numéro n'est jamais réattribué : le trou qu'elle laisse s'explique par cette trace.

En usage courant, deux voies font disparaître une écriture, et toutes deux sont nommées : sa **suppression** depuis sa fiche ou par l'API (depuis la v0.13.0), sous `journal_entry.deleted`, dans le cadre ci-dessus ; et la **dévalidation** d'une facture (depuis la v0.12.1), qui la repasse en brouillon et supprime l'écriture que sa validation avait créée. La seconde est une **décision de facturation** : elle est journalisée sous son propre nom, refusée sous neuf motifs — dont l'exercice clos ou suivi d'un exercice clôturé, le verrou de période, la contre-passation et l'envoi au client —, et elle ne touche jamais une écriture saisie à la main. *Hors usage courant, la restauration d'une sauvegarde remet les livres dans l'état de l'archive (§7.9).*

- `audit_log` alimenté à chaque création d'écriture, avec instantané (condition 1). *Il conserve aussi chaque modification d'écriture (`journal_entry.updated`, avant et après — celles d'avant la v0.12.0 comme celles depuis la v0.13.0), les suppressions (`journal_entry.deleted`, avec le contenu complet — celles d'avant la v0.12.0 comme celles depuis la v0.13.0), les dévalidations de factures (`invoice.unvalidated`), ainsi que les opérations des autres entités.*
- SHA-256 par fichier dans le `metadata.json` du ZIP d'export (condition 1 renforcée).
- Horodatage `created_at` pour le moment d'enregistrement (condition 2).
- Documentation des procédures dans ce manuel et dans le journal de bord (conditions 3 et 4).

! Deux réserves à connaître avant d'invoquer cette conformité

Le journal d'audit est inaltérable sur une installation en service, et il faut savoir à quoi cela tient. L'import d'une sauvegarde ne remplace plus la table : les entrées de l'archive sont **fusionnées** avec celles de l'installation, et chaque entrée porte le nom de son auteur au moment de l'écriture — il survit donc au remplacement des comptes utilisateurs. **! Cette fusion suppose que la sauvegarde soit celle de votre propre installation.** Importer celle d'une *autre* installation est techniquement possible, et les entrées du journal d'audit **déjà présentes chez vous** se retrouveraient alors rattachées à la société importée : le journal d'audit mêlerait deux comptabilités sans qu'aucun message ne le signale. N'importez que vos propres sauvegardes. *Une conséquence, rare, de cette fusion : la garde qui fige le paiement d'une facture fournisseur annulée lit le journal d'audit ; la fusion d'une archive d'une autre histoire peut, par coïncidence d'identifiants, figer une écriture saisie à la main sans rapport — elle reste contre-passable (limite assumée, issue #541).* La **réinitialisation des données de démonstration** efface encore la table, mais elle exige le rôle d'administrateur et est refusée dès que l'installation est finalisée. *Aucun verrou de base de données ne s'y oppose : la garantie est portée par l'application, non par le SGBD.* Reste, en mode test uniquement (`KESH_TEST_MODE`, jamais actif en déploiement réel), les points d'entrée de réinitialisation de la suite de tests automatisés.

L'empreinte SHA-256 est calculée et conservée par vous. Sans horodatage ni tiers indépendant, qui modifie les données recalcule l'empreinte. C'est une **bonne pratique d'archivage** — réelle, et supérieure à beaucoup de pratiques courantes — mais elle ne constitue pas à elle seule la garantie d'intégrité qu'un contrôle attend d'un support modifiable. Pour des livres devant résister à une contestation, complétez par un archivage tiers ou un horodatage qualifié.

i Note

La signature électronique qualifiée LSCSE (QES) n'est **pas requise** par l'OLiCo pour les exports comptables des PME (interprétation « p. ex. » du législateur). Le verdict (b)

« Dette explicite v0.2 » du document de recherche 9-5-4 confirme la conformité v0.1. Une implémentation QES est prévue en v0.2 via Epic 14 (issue GitHub #98).

9.8 Sécurité réseau (firewall)

Configuration firewall minimale (UFW sur Ubuntu/Debian) :

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp comment 'SSH'
sudo ufw allow 80/tcp comment 'HTTP (redirect to HTTPS)'
sudo ufw allow 443/tcp comment 'HTTPS'
sudo ufw enable
```

! Attention

UFW ne filtre pas un port **publié par Docker** (ports : d'un compose) : Docker place ses règles avant les siennes. Les compose fournis ne publient que le port de l'application ; ne publiez jamais celui de MariaDB (3306).

9.9 Mises à jour de sécurité du système hôte

- Appliquer les mises à jour OS automatiquement (unattended-upgrades sur Debian/Ubuntu).
- Surveiller les CVE Docker (mise à jour de la version Docker Engine régulière).
- Surveiller les CVE MariaDB (mise à jour de l'image MariaDB lors des releases mineures).
- Souscrire aux annonces de sécurité Kesh (label security sur les issues GitHub).

10 Monitoring et logs

10.1 Logs applicatifs

Par défaut, Kesh écrit ses logs sur `stdout` (capturés par Docker) et, en Docker, **aussi** dans un fichier avec rotation (§5.2). Le format du fichier est contrôlé par `KESH_LOG_FILE_FORMAT` (défaut `pretty` ; `json` disponible pour l'ingestion Loki/ELK). Consulter le flux `stdout` :

```
docker compose logs -f kesh-api
docker compose logs --tail=100 kesh
docker compose logs --since=1h kesh
```

10.2 Centralisation des logs

En production multi-instances, centraliser via :

- **ELK Stack** : Elasticsearch + Logstash + Kibana.
- **Loki + Grafana** : plus léger, recommandé pour les déploiements modestes.
- **Syslog distant** : configuration via le driver Docker logging `syslog`.

Exemple de driver logging Loki dans `docker-compose.yml` :

```
services:
  kesh:
    image: gcorbaz/kesh:latest
    logging:
      driver: loki
      options:
        loki-url: "https://loki.exemple.ch/loki/api/v1/push"
        loki-retries: "5"
        loki-batch-size: "400"
```

10.3 Supervision (état de santé)

Kesh n'expose pas d'endpoint de métriques Prometheus ni d'intégration Sentry en v0.x. La supervision repose sur deux mécanismes :

- **Healthcheck** `GET /health` — retourne `{"status":"ok","db":true,"version":...,"forgotPassword":...}`. Utilisé par le `healthcheck` Docker (redémarrage automatique si la DB est injoignable). Interrogeable par une sonde externe (Uptime Kuma, cron curl...).
- **Logs** avec rotation (§5.2) — pour l'analyse a posteriori et l'ingestion éventuelle dans un collecteur (Loki, ELK...).

10.4 Alerting recommandé

Alertes Prometheus / AlertManager suggérées :

- Taux d'erreurs HTTP 5xx > 1% sur 5 minutes.
- Latence p95 > 2 secondes sur les requêtes API.
- Connexion DB perdue (échec `kesh_db_queries_total`).
- Disque DB > 80% utilisé.
- Backup quotidien échoué (exit code non-zéro).

11 Conformité légale suisse

11.1 Code des obligations Art. 957a — Tenue de la comptabilité

Kesh satisfait intégralement les exigences de l'Art. 957a CO pour les PME suisses. Les détails de cette conformité sont documentés dans [_bmad-output/planning-artifacts/research-swiss-co-958f.md](#) (Story 9-5-4 « Recherche réglementaire Swiss CO Art. 957a / 958f »).

Synthèse :

- al. 1 : enregistrement intégral, fidèle et systématique → la saisie en partie double est contrôlée à trois niveaux (validation métier, contrainte de base, recalcul après écriture) ; une écriture enregistrée se **modifie** tant que son exercice est ouvert — sauf si l'une de ses lignes est lettrée, auquel cas elle se délettre d'abord —, chaque modification tracée avant et après ; elle devient **définitive** à la clôture — la sienne ou celle d'un exercice postérieur — ou sous le verrou de période, et se corrige alors par contre-passation ; et l'`audit_log` conserve l'historique — sous les deux réserves de la section *Conformité OLICo Art. 9*.
- al. 2 : principe de régularité → instancié par les patterns techniques Kesh (clarté multilingue, justification, traçabilité).
- al. 3 : pièce comptable papier ou électronique → support électronique autorisé.
- al. 4 : monnaie nationale ou monnaie principale → CHF par défaut, multi-monnaies prévu v0.2.
- al. 5 : langues nationales ou anglais → i18n FR/DE/IT/EN actif.

11.2 Code des obligations Art. 958f — Conservation 10 ans

- al. 1 : conservation 10 ans à compter de la fin de l'exercice → garanti par la persistance MariaDB (sous réserve d'admin DB + backup approprié).
- al. 2 : rapport de gestion imprimé et signé → **responsabilité de la PME utilisatrice**, hors scope logiciel.
- al. 3 : support libre + lien garanti + lisibilité possible → garanti par `audit_log` + SHA-256 + format CSV ouvert du ZIP d'export. **Depuis la v0.12.1, le journal d'audit VOYAGE avec l'export** (`audit_log.csv`, issue #386) : il n'est plus seulement conservé dans l'instance, il part avec les livres.

11.3 Rétention des rappels débiteurs (dossier de recouvrement)

Depuis la v0.7, chaque rappel envoyé (par e-mail ou enregistré comme rappel papier) est **historisé** dans la table `invoice_reminders` avec une **copie du texte réclamé** et des **frais snapshotés** au moment de l'envoi. Cet historique constitue une **preuve de recouvrement** : en cas de poursuite (LP), il documente les relances effectuées, leurs dates et le destinataire.

- **Conservation** : les rappels suivent la rétention comptable de 10 ans (CO Art. 958f al. 1), portée par la persistance MariaDB (sous réserve d'une administration DB et d'une sauvegarde appropriées).
- **Export et sauvegarde** : les trois tables du dispositif de rappels (`dunning_levels`, `company_dunning_settings`, `invoice_reminders`) figurent dans l'**export de souveraineté** (`exports/global.zip`) et dans la **sauvegarde administrateur** — l'absence des rappels de l'export serait une perte de valeur probatoire.
- **Donnée personnelle (LPD)** : le champ `sent_to` (adresse e-mail du débiteur au moment de l'envoi) est une donnée personnelle conservée pour le respect d'une **obligation légale** (tenue de la comptabilité et preuve de recouvrement, CO Art. 958f), base de licéité admise par la LPD. Elle est conservée 10 ans au même titre que les autres pièces comptables.

! Attention

L'historique des rappels est protégé par l'application, et non plus seulement par votre vigilance. Une facture qui a fait l'objet de relances ne peut plus être **dévalidée** — la dévalidation supprimerait son écriture, et la suppression du brouillon qui s'ensuivrait effacerait le texte réclamé, les frais et le destinataire. Kesh refuse, en nommant ce motif (garde livrée par l'issue #260, puis reprise par la dévalidation). Pour annuler une facture déjà relancée, la voie reste l'**avoir** : l'historique demeure complet, et c'est ce que la valeur probatoire exige.

11.4 Ordonnance OLICo (RS 221.431)

- Art. 1 : journal chronologique → implémenté (rapport *Journaux*). **Le grand livre — l'extrait d'un compte donné — n'existe pas encore** : aucun rapport ne le produit, et la liste des écritures ne se filtre pas par compte. C'est la première pièce que demande une révision ; elle doit aujourd'hui être reconstituée depuis l'export CSV.
- Art. 3 : intégrité (modification apparente) → les corrections se font par **contre-passation**, qui laisse l'écriture d'origine en place, ou — tant que l'exercice est ouvert, et après délettrage si l'une de ses lignes est lettrée — par **modification tracée**, le journal d'audit conservant l'écriture avant et après. *La couverture du journal n'est pas encore complète, mais la réserve a changé d'objet : la gestion des utilisateurs, la modification de la société et les pièces importées y sont désormais inscrites. Ce qui manque est la remise à zéro des données de*

démonstration (issue #434) et les gestes de session (issue #435).

- Art. 9 al. 1.b : supports modifiables (disque dur, SSD) → conditions 1-4 satisfaites (cf. section sécurité plus haut).
- Art. 10 : contrôle et migration → test de restauration annuel obligatoire (cf. section Sauvegarde).

11.5 LSCSE (signature électronique qualifiée)

La signature électronique qualifiée n'est **pas obligatoire** pour la conservation comptable PME selon l'OLICo Art. 9 (« p. ex. signature électronique »). Une implémentation QES est prévue en v0.2 (Epic 14, issue #98) pour les PME souhaitant un niveau supplémentaire de défense.

11.6 Loi sur la TVA (LTVA)

Kesh comptabilise la TVA suisse : TVA due sur les ventes (générée automatiquement à la validation des factures), impôt préalable sur les achats, décompte par période (solde net dû à l'AFC) et contrôle de cohérence entre le décompte et les écritures des factures qu'il retient. La méthode effective (taux réels) est seule supportée ; le taux de la dette fiscale nette (TDFN) ne l'est pas. **Le décompte n'est pas rapproché du solde du compte de TVA due** : ce rapprochement, qu'exige la concordance annuelle (art. 72 al. 1 LTVA), reste à la charge de l'utilisateur.

Configuration des comptes TVA. Trois comptes TVA standard du plan comptable suisse sont nécessaires à la comptabilisation et se configurent depuis *Paramètres* → *Facturation* :

- **TVA due** (p. ex. 2200, passif) — TVA collectée sur les ventes.
- **Impôt préalable** (p. ex. 1171, actif) — TVA récupérable sur les achats. À ne pas confondre avec l'impôt anticipé (1170/Verrechnungssteuer), qui est un compte distinct.
- **Décompte TVA** (p. ex. 2206, passif) — solde net vis-à-vis de l'AFC.

Ces comptes sont ajoutés automatiquement aux plans comptables (PME, indépendant, association) ; les installations existantes sont complétées sans modifier les comptes déjà utilisés. Chacun doit être un compte actif du bon type et **imputable** (pas un compte de regroupement, de résultat ou de clôture) : un compte non imputable est refusé au moment où vous le désignez — un compte déjà désigné, devenu non imputable depuis, ne bloque pas l'enregistrement des autres réglages. Il bloque en revanche **les écritures qui l'utilisent** : un compte *TVA due* devenu non imputable fait refuser la validation d'une facture portant de la TVA, et un compte *Impôt préalable* devenu non imputable, la saisie (ou la complétion) d'une facture fournisseur portant de la TVA — avec un message qui nomme le compte et renvoie à ces paramètres, où il faut le remplacer par un compte imputable. Tant que le compte *TVA due* n'est pas configuré, la validation d'une facture portant de la TVA est refusée (message « configuration requise »). L'usage côté comptable (décompte, réconciliation) est décrit dans le *manuel utilisateur*, section

Décompte TVA.

Compte créanciers. La même page, section *Comptes par défaut*, désigne aussi le **compte créanciers** (dette fournisseurs, passif ; 2000 dans les plans livrés, désigné d'office à la création d'une société), que crédite la saisie d'une facture fournisseur. Il obéit au même principe : un compte **actif et imputable**, refusé non imputable à la désignation ; devenu non imputable depuis, il reste affiché dans les paramètres et fait refuser la saisie et la complétion des factures fournisseurs jusqu'à ce qu'un administrateur le remplace. Il en va de même pour le compte de *créance client* : devenu non imputable, il fait refuser la validation des factures. **Après une mise à jour vers la v0.13.0**, vérifiez que ces comptes sont imputables. Les *avoirs* ne sont pas soumis à ce contrôle.

Compte de différences d'arrondi. Comme le veut l'usage suisse, le **total d'une facture émise est arrondi à 5 centimes** (123.44 devient 123.45, 234.52 devient 234.50), avec une ligne « Arrondi » sur la facture ; ce réglage est **actif par défaut** et se désactive dans la même section, case « *Arrondir le total des factures émises à 5 centimes* » — les factures déjà émises gardent leur total. L'écart, sans TVA, est écrit **à la validation** et reste attaché à la facture ; un avoir le reprend et l'annule. Changer de compte d'arrondi ne déplace pas les factures déjà émises : leur avoir contre-passe l'arrondi sur le compte qu'elles ont mouvementé. Gardez donc l'ancien compte actif tant qu'un avoir peut les viser (sinon : le réactiver, émettre l'avoir, le ré-archiver). Par ailleurs, une facture émise sans arrondi dont le total porte plus de deux décimales — quantité ou prix unitaire fractionnaire — laisse, une fois payée au centime, un écart d'un demi-centime au plus ; et le *solde du reste* d'une telle facture y passe sa fraction de centime, quelle que soit la nature choisie — d'où l'exigence de ce compte pour solder un reste à fraction de centime. Ces écarts vont au **compte de différences d'arrondi**, désigné depuis *Paramètres* → *Facturation*, section *Différences d'arrondi*. Ce doit être un compte de **charge ou de produit**, actif et **imputable** (pas un compte de regroupement). Les plans livrés (PME, indépendant, association) le proposent : 6940 *Différences d'arrondi*, une charge, **désignée d'office** à la création d'une société. Pour une société créée auparavant, ou si vous préférez un autre compte : créez-le au besoin dans le plan comptable, puis choisissez-le dans les paramètres. **Sans ce compte, une facture dont le total n'est pas déjà un multiple de 5 centimes ne peut pas être validée** : la validation est refusée, avec un message qui renvoie aux paramètres. Pour une société créée avant cette version, désignez-le donc avant d'émettre.

Comptes du solde du reste. La même page, section *Solde du reste*, désigne les comptes qui recevront ce qui reste dû quand une facture est soldée sans être entièrement payée, selon la **nature** de l'écart : l'**escompte accordé** au client, les **frais bancaires** retenus par sa banque, la **perte sur un débiteur**. Chacun doit être un compte de **charge ou de produit**, actif et **imputable**. Les plans livrés les proposent, **désignés d'office** à la création d'une société : 3800 pour l'escompte (*Déductions sur ventes* pour une PME, *Déductions sur prestations* pour un indépendant), 6900 *Charges financières* pour les frais bancaires, et 3805 *Pertes sur créances*, ajouté aux trois plans, pour les pertes. Le plan des **associations** ne propose **aucun compte**

d'escompte — son 3800 est *Autres produits* — : une association qui accorde des escomptes choisit elle-même le compte. Pour une société créée avant cette version, les trois réglages sont vides : créez au besoin les comptes dans le plan comptable, puis choisissez-les ici. Un solde du reste est **refusé**, sans rien écrire, si le compte de la nature, le compte de différences d'arrondi ou le compte de TVA due est le **compte débiteurs de la facture** — le cas d'un compte débiteurs changé de type depuis, puis désigné ici : désignez alors un autre compte.

Montant minimum d'une facture. La même page, section *Montant minimum*, fixe un seuil sous lequel une facture ne peut pas être validée (comparé au total arrondi). Vide par défaut : aucun seuil. Les avoirs n'y sont pas soumis.

Quand un paiement — règlement manuel ou rapprochement bancaire — **égale le reste dû arrondi au centime** alors que le reste exact en diffère (10.01 pour 10.0050), il **solde** la facture : le règlement est enregistré pour le reste exact, et l'écart passe en **troisième ligne** d'écriture sur le compte de différences d'arrondi — au crédit si le paiement dépasse le reste exact, au débit sinon. La créance se ferme ainsi à zéro. Si aucun compte utilisable n'est désigné — réglage vide, ou compte archivé, rendu non imputable ou changé de type depuis, ou devenu le compte débiteurs de la facture —, un tel paiement est **refusé sans rien écrire**, avec un message qui renvoie à *Paramètres* → *Facturation*. Un paiement sans écart n'exige aucun réglage. Un règlement manuel se saisit au centime : un montant à plus de deux décimales est refusé.

Comptes débiteurs et créanciers. Sur la même page, section *Comptes par défaut*, le **compte débiteurs** (créance client, actif ; 1100 dans les plans livrés) reçoit la créance de chaque facture validée, et le **compte créanciers** (présenté plus haut) la dette de chaque facture fournisseur ; chacun est soldé par les règlements. Ni l'un ni l'autre ne peut être le compte du grand livre **lié à un compte bancaire** : un encaissement ou un paiement par ce compte bancaire écrirait le même compte au débit et au crédit — une écriture nulle. Kesh refuse donc de désigner ici un compte lié à un compte bancaire non archivé (le message nomme le compte bancaire), et réciproquement, dans *Administration* → *Comptes bancaires*, de lier un compte bancaire au compte débiteurs ou créanciers désigné ; les menus ne les proposent pas. Un lien de cette forme **antérieur à la v0.13.0** n'est pas défaut : il ne bloque pas l'enregistrement des autres réglages, mais chaque règlement et chaque rapprochement de facture par ce compte bancaire sont refusés — reliez alors le compte bancaire à son propre compte de banque.

Le format de décompte officiel AFC (e-décompte ESTV) et la conformité LTVA Art. 70 (factures électroniques signées) restent hors périmètre à ce stade.

11.7 Documentation à conserver

Conformément à OLICo Art. 9 al. 1.b ch. 4 (« procédures et modes d'utilisation consignés »), conserver dans votre archive comptable :

- Une copie de ce manuel administrateur (version utilisée).
- Une copie du manuel utilisateur.

- Une copie du document de recherche réglementaire ([research-swiss-co-958f.md](#)).
- Les procès-verbaux de test de restauration annuels.
- Le procès-verbal de migration (cf. metadata.json des ZIP de souveraineté).

! Astuce

Une revue juridique externe (avocat suisse spécialisé en droit commercial / IT) est recommandée mais non bloquante pour une publication commerciale. Coût ordre-de-grandeur : CHF 1'000–3'000 pour une PME. Voir le document de recherche §Verdict pour les détails.

12 Dépannage

12.1 Kesh ne démarre pas

Symptômes Le conteneur `kesh-api` sort immédiatement après `docker compose up`, ou `docker compose ps` montre `kesh-api` en `Restarting`.

```
docker compose logs kesh-api | tail -50
```

Causes courantes :

- **Compose refuse de démarrer** ([docker-compose.yml](#), depuis la 0.13.0) : `required variable MARIADB_PASSWORD is missing a value` (ou `MARIADB_ROOT_PASSWORD`), affiché **au terminal** de la commande — aucun conteneur ne démarre, les journaux n'en disent rien. Compose ne nomme qu'une variable à la fois, pas toujours la même : posez les deux dans `.env` — sur une installation existante, les valeurs avec lesquelles la base a été créée (§8.3.1).
- **Erreur 1045** (« Access denied for user ... ») dans les journaux de `kesh-api`, suivie d'un **indice** : MariaDB refuse le mot de passe de `DATABASE_URL`. Avec [docker-compose.yml](#), presque toujours un `MARIADB_PASSWORD` changé dans `.env` alors que la base garde celui de sa création : revenir à l'ancienne valeur, puis suivre le §8.3.2.
- **DATABASE_URL invalide** : vérifier le format `mysql://user:pass@host:port/db`.
- **DB pas encore prête** : ajouter un `depends_on` avec `condition: service_healthy` dans [docker-compose.yml](#).
- **KESH_JWT_SECRET trop court** : doit faire au moins 32 octets (ex. `openssl rand -hex 32` = 64 caractères hexadécimaux).

- **Migrations DB échouées** : voir log détaillé. Solution : restaurer depuis sauvegarde et investiguer.

12.2 Erreurs 401 en cascade côté frontend

Diagnostic

Symptômes Plusieurs appels API consécutifs échouent en 401 `Unauthorized` alors que l'utilisateur est censé être authentifié.

Cause Refresh token expiré ou invalidé (cf. KF #54 « E2E test helpers cascade 401 », fermée Epic 9.5).

Solution L'utilisateur doit se reconnecter. Si le problème persiste pour tous les utilisateurs, vérifier l'horloge système (drift NTP).

12.3 Performance lente sur de gros datasets

Symptômes Génération de rapports ou exports ZIP > 30 secondes sur une société de plus de 50'000 écritures.

```
docker compose exec -T mariadb sh -c 'mariadb -u root \  
-p"$MARIADB_ROOT_PASSWORD" -e "SELECT COUNT(*) FROM kesh.journal_entries  
GROUP BY company_id;"'
```

Solutions

- Augmenter `innodb_buffer_pool_size` (cf. config MariaDB).
- Vérifier que les index FULLTEXT sont en place (Story 7-4).
- Surveiller `slow.log` et optimiser les requêtes lentes éventuelles.
- Considérer une migration vers MariaDB 11.5+ qui inclut des optimisations supplémentaires.

12.4 Reset du mot de passe administrateur (break-glass)

Si l'administrateur a perdu son mot de passe et qu'il n'y a pas d'autre administrateur, utilisez le mécanisme **break-glass** par variables d'environnement (il n'existe pas d'outil en ligne de commande dédié). Dans le fichier `.env`, renseignez `KESH_ADMIN_USERNAME` avec le nom de l'admin existant et `KESH_ADMIN_PASSWORD` avec le nouveau mot de passe (min. 12 caractères), puis redémarrez :

```
docker compose up -d kesh-api # relit .env au démarrage
```

Au démarrage, si un admin portant ce nom d'utilisateur existe déjà, son mot de passe est réinitialisé sur la nouvelle valeur. Retirez ensuite `KESH_ADMIN_PASSWORD` du `.env`, puis `docker compose up -d kesh-api`. Si le recovery par email est activé (`KESH_FEATURE_FORGOT_PASSWORD=true`), l'utilisateur peut aussi passer par le lien « Mot de passe oublié ».

12.5 Données corrompues détectées

Si une vérification d'intégrité (SHA-256 d'un export ZIP ne correspond plus, `audit_log` incohérent, etc.) révèle une corruption :

1. **Arrêter immédiatement** l'instance : `docker compose stop kesh-api`.
2. Conserver une copie de l'état actuel : `tar czf kesh-corrupted-$(date +%F).tar.gz /opt/kesh`.
3. Restaurer depuis la dernière sauvegarde saine (vérifier le SHA-256 de la sauvegarde elle-même avant restauration).
4. Analyser la cause racine : intrusion sécurité ? corruption disque ? bug applicatif ?
5. Documenter l'incident pour l'audit OLICo.

X Danger

Une corruption de données comptables peut avoir des implications légales sévères. Ne pas tenter de « réparer » manuellement les données corrompues — restaurer depuis sauvegarde et investiguer hors production.

13 Maintenance

13.1 Rotation des logs

Configurer logrotate pour éviter que les logs Docker ne saturent le disque :

```
# /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "5"
  }
}
```

Puis redémarrer Docker : `sudo systemctl restart docker`.

13.2 Maintenance MariaDB

Tâches périodiques recommandées :

Tâche	Description	Fréquence
OPTIMIZE TABLE	Défragmentation des tables InnoDB.	Trimestrielle
ANALYZE TABLE	Mise à jour des statistiques d'optimiseur.	Mensuelle
Vérification cohérence	<code>mariadb-check --all-databases</code> .	Mensuelle
Purge binlog	Suppression des binary logs expirés (auto via <code>expire_logs_days</code>).	Quotidienne (auto)

13.3 Vérification périodique de l'intégrité (OLICo Art. 10)

Conformément à OLICo Art. 10 al. 1, vérifier l'intégrité des supports d'information régulièrement :

```
#!/bin/bash
# /usr/local/bin/kesh-integrity-check.sh
# Verification mensuelle de l'integrite des sauvegardes

BACKUP_DIR="/var/backups/kesh"
LOG_FILE="/var/log/kesh-integrity.log"

echo "  Verification integrite $(date +%F)  " >> "${LOG_FILE}"

cd "${BACKUP_DIR}"
for backup in kesh_*.sql.gz; do
    if sha256sum -c "${backup}.sha256" >> "${LOG_FILE}" 2>&1; then
        echo "  ${backup} OK" >> "${LOG_FILE}"
    else
        echo "  ${backup} INTEGRITE COMPROMISE - ALERTE" >> "${LOG_FILE}"
        # Envoyer une alerte (mail, Slack, PagerDuty, etc.)
    fi
done
```

14 Annexes

14.1 Liste complète des variables d'environnement

Voir section 5.1.

14.2 Opérations en ligne de commande

Kesh ne fournit pas d'outil CLI d'administration dédié : le binaire `kesh-api` est le seul exécutable, et il applique automatiquement les migrations DB au démarrage. Les opérations d'administration passent par l'**interface web** (gestion des utilisateurs, sauvegarde/restauration, export global) ou par les mécanismes documentés ailleurs dans ce manuel :

- **Reset admin** : mécanisme break-glass par `.env` (§12.4).
- **Migrations** : appliquées automatiquement au boot ; vérifiables via les logs de démarrage.
- **Export / import complet** : écran *Administration* → *Sauvegarde complète* / *Restaurer* (format `.keshbackup`).
- **Sauvegarde DB brute** : `mariadb-dump` — sur le conteneur MariaDB du `docker-compose.yml` standard (§7.4) ; sur Synology, par une tâche planifiée qui précède Hyper Backup (§7.8), lequel copie ensuite le dump, non la base.

14.3 Référence des ports

Service	Port	Notes
Frontend HTTPS (utilisateur)	443	Via reverse proxy
API Axum (interne)	80	Localhost uniquement
MariaDB	3306	Non publié (<code>docker-compose.yml</code>)

14.4 Glossaire

CAMT.053 Standard ISO 20022 d'extrait de compte bancaire suisse.

Company Entité comptable (PME, association) dans Kesh ; isolation multi-tenant par `company_id`.

Fiscal year Exercice comptable, généralement annuel (1^{er} janvier – 31 décembre par défaut).

IDOR Insecure Direct Object Reference, classe d'attaque évitée via le scoping `company_id`.

Journal d'audit (angl. audit log) Trace des actions métier (table `audit_log`), inaltérable sur

une installation en service. *La couverture n'est pas totale : voir les réserves de la section « Conformité ».*

Journal entry Écriture comptable en partie double.

OLICo Ordonnance sur la tenue et la conservation des livres de comptes (RS 221.431).

pain.001 Standard ISO 20022 d'ordre de paiement (Customer Credit Transfer Initiation).

QES Qualified Electronic Signature (LSCSE Art. 2 let. e).

QR Bill Standard suisse 2020+ de facture avec QR code (SIX).

RBAC Role-Based Access Control.

Refresh token Token longue durée pour renouveler un access token JWT.

Tenant Synonyme de *company* dans un contexte multi-tenant.

14.5 Liens externes

- Dépôt GitHub Kesh
- Code des obligations RS 220
- Ordonnance OLICo RS 221.431
- Loi LSCSE RS 943.03
- Guide PME comptabilité (SECO)
- QR Bill SIX
- ISO 20022 (CAMT.053, pain.001)

14.6 Support et communauté

- Issues GitHub : <https://github.com/guycorbaz/kesh/issues>
- Discussions : <https://github.com/guycorbaz/kesh/discussions>
- Documentation technique : <https://github.com/guycorbaz/kesh/tree/main/docs>

i Note**À propos de cette version**

Ce document décrit Kesh version **0.12.1** (cible release : v0.12, publié le 2026-10-07). Les manuels Kesh sont mis à jour à chaque release. Pour la dernière version, consultez <https://github.com/guycorbaz/kesh>.

Si vous identifiez une incohérence entre ce manuel et le comportement effectif du logiciel, merci d'ouvrir une issue sur GitHub avec le template `bug_report.yml`.